

Character Frequency

1. Overview

Character frequency means counting how many times each character appears in a string. It is one of the most common building blocks for string interview questions.

2. Intuition

Think of every character as a bucket. When you see a character, put one coin into its bucket.

Why it works:

- Equal strings have equal frequency counts.
- Anagrams have equal frequency counts.
- Missing or extra characters are detected by count differences.
- For lowercase English letters, only 26 buckets are needed.

3. When to Use It

Use character frequency when the problem says:

- "count characters"
- "can be rearranged"
- "anagram"
- "minimum deletions to make..."
- "first unique character"
- "compare two strings ignoring order"
- "frequency of each letter"

4. When Not to Use It

Avoid it when:

- Character order matters, such as subsequence or substring matching.
- You need exact pattern positions; use KMP, Z, or Rabin-Karp.
- The alphabet is huge and sparse; use `unordered_map` instead of a fixed array.
- You assume only lowercase letters but input contains uppercase, digits, or Unicode.

5. Core Concepts

Concept	Meaning	Why It Matters
Fixed alphabet	Known character set like <code>a-z</code>	Allows <code>vector<int>(26)</code>
Dynamic alphabet	Unknown character set	Use <code>unordered_map<char, int></code>
Frequency difference	Add for one string, subtract for another	Useful for anagram checks
Unique character	Frequency is exactly 1	Used in first non-repeating character

Example: **banana**

char	count
b	1
a	3
n	2

6. Step-by-Step Algorithm

1. Create a frequency array or hash map.
2. Traverse the string from left to right.
3. For every character, increase its count.
4. Use the counts to answer the required question.

7. Dry Run

Input: **s = "aabbcc"**

Step	Character	Count Updates
1	a	a = 1
2	a	a = 2
3	b	b = 1
4	b	b = 2
5	c	c = 1

Final answer: **a:2, b:2, c:1**

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

vector<int> characterFrequencyLowercase(const string& s) {
    vector<int> freq(26, 0);

    for (char ch : s) {
        if (ch >= 'a' && ch <= 'z') {
            freq[ch - 'a']++;
        }
    }

    return freq;
}

bool areAnagramsLowercase(const string& a, const string& b) {
```

```

    if (a.size() != b.size()) return false;

    vector<int> freq(26, 0);
    for (char ch : a) freq[ch - 'a']++;
    for (char ch : b) freq[ch - 'a']--;

    for (int count : freq) {
        if (count != 0) return false;
    }
    return true;
}

int main() {
    string s = "aabbcc";
    vector<int> freq = characterFrequencyLowercase(s);

    for (int i = 0; i < 26; i++) {
        if (freq[i] > 0) {
            cout << char('a' + i) << " " << freq[i] << "\n";
        }
    }

    cout << boolalpha << areAnagramsLowercase("listen", "silent") << "\n";
    return 0;
}

```

pYTHON IMPLEMENTATION

```

from collections import Counter

def character_frequency(s: str) -> dict[str, int]:
    return dict(Counter(s))

def are_anagrams(a: str, b: str) -> bool:
    return Counter(a) == Counter(b)

if __name__ == "__main__":
    print(character_frequency("aabbcc"))
    print(are_anagrams("listen", "silent"))

```

10. Code Explanation

- `freq[ch - 'a']` maps lowercase characters to indices `0..25`.
- The anagram function increments counts for the first string and decrements for the second.
- If every count becomes zero, both strings contain the same multiset of characters.

- The C++ code guards the frequency function against non-lowercase characters, but the anagram helper assumes lowercase input.

11. Complexity Analysis

Operation	Time	Space
Count fixed alphabet	$O(n)$	$O(1)$
Count arbitrary chars	$O(n)$ average	$O(k)$
Anagram check	$O(n)$	$O(1)$ for lowercase

n is string length, k is number of distinct characters.

12. Common Patterns

Pattern	How to Identify	Approach	Examples
Anagram equality	"rearrange", "same letters"	Compare frequencies	LeetCode Valid Anagram
First unique	"first non-repeating"	Count, then scan original string	LeetCode First Unique Character
Can form target	"ransom note", "magazine"	Subtract required counts	LeetCode Ransom Note

13. Common Mistakes

- Forgetting to handle uppercase letters.
- Using 26 buckets when input includes digits or symbols.
- Sorting when counting is enough.
- Forgetting to compare string lengths for anagrams.
- Treating Unicode characters as single `char` values in C++.

14. Edge Cases

- Empty string.
- Single character.
- All characters same.
- All characters unique.
- Uppercase and lowercase mixed.
- Spaces and punctuation.

15. Variations

Variation	What Changes	Importance
Lowercase fixed array	Use 26 buckets	Very important for placements

Variation	What Changes	Importance
ASCII array	Use 128 or 256 buckets	Useful in OAs
Hash map frequency	Unknown alphabet	Important in practical problems
Frequency difference	Add/subtract between strings	Very common

16. Related Algorithms/Data Structures

- Hash map: use when alphabet is not fixed.
- Sliding window: combines frequency counts with moving substring windows.
- Trie: better for prefix queries over many strings.
- Sorting: simpler for tiny anagram problems but slower at $O(n \log n)$.

17. Practice Problems

Easy

Problem	Platform	Main Idea	Difficulty
Valid Anagram	LeetCode	Compare character frequencies	Easy
Ransom Note	LeetCode	Check supply counts	Easy

Medium

Problem	Platform	Main Idea	Difficulty
Group Anagrams	LeetCode	Frequency signature	Medium
Sort Characters By Frequency	LeetCode	Count and sort by count	Medium
Minimum Deletions to Make Character Frequencies Unique	LeetCode	Frequency of frequencies	Medium

Hard

Problem	Platform	Main Idea	Difficulty
Minimum Window Substring	LeetCode	Frequency + sliding window	Hard
Find All Anagrams in a String	LeetCode	Window frequency	Medium/Hard in OAs

18. Interview Explanation

"I use a frequency array or hash map to count occurrences of each character. For fixed lowercase strings, a 26-size array gives linear time and constant space. This is useful whenever order does not matter, such as anagrams, unique characters, or checking whether one string can be formed from another."

19. Revision Notes

- Key idea: count characters into buckets.
- Lowercase formula: `index = ch - 'a'`.
- Use `unordered_map` for unknown character sets.
- Complexity: $O(n)$ time.
- Trap: frequency ignores order.

20. Final Cheat Sheet

Item	Note
Use when	Counts, anagrams, uniqueness, rearrangement
Main operation	Increment/decrement character counts
Complexity	$O(n)$ time, $O(1)$ or $O(k)$ space
Key code idea	<code>freq[ch - 'a']++</code>
Edge cases	Empty string, mixed case, non-letter chars

Two Pointers on Strings

1. Overview

Two pointers use two indices to scan a string from different positions. The pointers may move from both ends toward the center, or both move in the same direction.

2. Intuition

Imagine checking a word with two fingers: one at the start and one at the end. Compare or move them based on the problem condition.

Why it works:

- Many string questions depend on relationships between characters at two positions.
- Moving pointers avoids unnecessary nested loops.
- Each pointer usually moves at most n times.

3. When to Use It

Use it for:

- "reverse a string"
- "valid palindrome"
- "remove characters from both ends"
- "compare two strings with backspaces"
- "merge strings"
- "minimum operations from ends"
- sorted-like character constraints

4. When Not to Use It

Avoid it when:

- The problem needs remembering many past characters; use hashing or DP.
- You need all substring frequencies; use sliding window.
- The relation is not monotonic, so pointer movement has no clear rule.
- You need pattern matching; use KMP, Z, or Rabin-Karp.

5. Core Concepts

Concept	Meaning	Example
Opposite pointers	<code>left = 0, right = n - 1</code>	Palindrome, reversal
Same-direction pointers	Slow and fast indices	Filtering characters
Movement rule	Decide which pointer moves	Skip non-alphanumeric
Termination	Stop when pointers cross	<code>left >= right</code>

6. Step-by-Step Algorithm

1. Initialize one pointer at the start and one at the end.
2. While the pointers have not crossed:
3. Compare or process the characters.
4. Move one or both pointers according to the problem rule.
5. Return the final result.

7. Dry Run

Check palindrome for `s = "racecar"`.

Step	left	right	Compare	Action
1	0	6	<code>r == r</code>	move inward
2	1	5	<code>a == a</code>	move inward
3	2	4	<code>c == c</code>	move inward
4	3	3	center	stop

Final answer: palindrome.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

bool isPalindromeIgnoringNonAlnum(const string& s) {
    int left = 0;
```

```

int right = (int)s.size() - 1;

while (left < right) {
    while (left < right && !isalnum((unsigned char)s[left])) left++;
    while (left < right && !isalnum((unsigned char)s[right])) right--;

    if (tolower((unsigned char)s[left]) != tolower((unsigned char)s[right])) {
        return false;
    }

    left++;
    right--;
}

return true;
}

void reverseStringInPlace(string& s) {
    int left = 0, right = (int)s.size() - 1;
    while (left < right) {
        swap(s[left], s[right]);
        left++;
        right--;
    }
}

int main() {
    string s = "A man, a plan, a canal: Panama";
    cout << boolalpha << isPalindromeIgnoringNonAlnum(s) << "\n";

    string t = "placement";
    reverseStringInPlace(t);
    cout << t << "\n";
    return 0;
}

```

pYTHON IMPLEMENTATION

```

def is_palindrome_ignoring_non_alnum(s: str) -> bool:
    left, right = 0, len(s) - 1

    while left < right:
        while left < right and not s[left].isalnum():
            left += 1
        while left < right and not s[right].isalnum():
            right -= 1

        if s[left].lower() != s[right].lower():
            return False

        left += 1

```

```
        right -= 1

    return True

def reverse_string(s: str) -> str:
    chars = list(s)
    left, right = 0, len(chars) - 1
    while left < right:
        chars[left], chars[right] = chars[right], chars[left]
        left += 1
        right -= 1
    return "".join(chars)
```

10. Code Explanation

- `left` and `right` mark the current pair of characters.
- Inner `while` loops skip characters that should not participate.
- `tolower` makes comparison case-insensitive.
- Reversal swaps symmetric characters until the middle is reached.

11. Complexity Analysis

Operation	Time	Space
Palindrome check	$O(n)$	$O(1)$
In-place reversal	$O(n)$	$O(1)$

Each pointer moves at most n times.

12. Common Patterns

Pattern	Identify It	Approach	Examples
Palindrome	Compare ends	Opposite pointers	Valid Palindrome
Reverse	Swap ends	Opposite pointers	Reverse String
Filter in place	Keep valid chars	Slow-fast pointers	Remove Duplicates-like string variants

13. Common Mistakes

- Forgetting `left < right` checks inside skip loops.
- Moving only one pointer after a successful comparison.
- Using `char` directly with `isalnum` on negative values in C++; cast to `unsigned char`.
- Assuming two pointers always works without a monotonic movement rule.

14. Edge Cases

- Empty string.

- One character.
- All punctuation.
- Mixed uppercase/lowercase.
- Even and odd length strings.

15. Variations

Variation	Use
Opposite pointers	Palindrome, reversal
Slow-fast pointers	Remove or compress characters
Multiple pointers	Merge or compare transformed strings

16. Related Algorithms/Data Structures

- Sliding window: same-direction pointers with a maintained window.
- Stack: useful for backspace/string reduction problems.
- Hashing: useful when comparing substrings instead of characters.

17. Practice Problems

Easy

Problem	Platform	Main Idea	Difficulty
Valid Palindrome	LeetCode	Skip invalid chars and compare	Easy
Reverse String	LeetCode	Swap using two pointers	Easy

Medium

Problem	Platform	Main Idea	Difficulty
Valid Palindrome II	LeetCode	Try deleting one mismatch	Medium
Reverse Words in a String	LeetCode	Reverse parts carefully	Medium
Backspace String Compare	LeetCode	Reverse scan or stack	Medium

Hard

Problem	Platform	Main Idea	Difficulty
Minimum Window Subsequence	LeetCode	Two scans	Hard
Shortest Palindrome	LeetCode	Two-pointer idea plus KMP	Hard

18. Interview Explanation

"Two pointers let me compare or process characters from two positions without nested loops. For palindrome-style problems I start at both ends and move inward. The correctness comes from discarding already-verified characters, and the complexity is linear because each pointer moves only across the string once."

19. Revision Notes

- Opposite pointers: $l = 0, r = n - 1$.
- Stop at $l \geq r$.
- Move both after a match.
- Good for palindrome/reversal.
- Trap: no clear movement rule means two pointers may not apply.

20. Final Cheat Sheet

Item	Note
Use when	Compare ends, reverse, scan with two indices
Main operation	Move pointers based on condition
Complexity	Usually $O(n)$ time, $O(1)$ space
Key code idea	<code>while (left < right)</code>
Edge cases	Empty, one char, skipped chars, odd length

Sliding Window on Strings

1. Overview

Sliding window maintains a contiguous substring using two pointers, usually `left` and `right`, while tracking useful information such as character counts.

2. Intuition

Imagine a window moving over the string. You expand it by moving `right`, and shrink it by moving `left` when the window becomes invalid.

Why it works:

- Many substring problems ask for longest, shortest, or count of valid contiguous segments.
- Instead of rebuilding every substring, update the current window incrementally.
- Each character enters and leaves the window at most once.

3. When to Use It

Trigger phrases:

- "longest substring"
- "smallest substring"
- "at most k distinct"

- "without repeating characters"
- "contains all characters"
- "find all anagrams"
- "contiguous substring"

4. When Not to Use It

Avoid it when:

- Subsequence is involved, not substring.
- The condition is not maintainable by adding/removing endpoints.
- You need arbitrary pattern matching; use KMP/Z/Rabin-Karp.
- You need non-contiguous choices; use DP or greedy.

5. Core Concepts

Concept	Meaning	Why It Matters
Window	<code>s[left..right]</code>	Current substring
Expand	Move <code>right</code>	Adds a new character
Shrink	Move <code>left</code>	Restores validity
Invariant	Condition kept true	Makes answer update reliable

6. Step-by-Step Algorithm

1. Set `left = 0`.
2. Iterate `right` from `0` to `n - 1`.
3. Add `s[right]` to the window state.
4. While the window violates the condition, remove `s[left]` and increment `left`.
5. Update the answer using the valid window.

7. Dry Run

Longest substring without repeating characters for `s = "abcabcbb"`.

right	char	Window After Fix	Answer
0	a	a	1
1	b	ab	2
2	c	abc	3
3	a	bca	3
4	b	cab	3
5	c	abc	3
6	b	cb	3

right	char	Window After Fix	Answer
7	b	b	3

Final answer: 3.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

int lengthOfLongestSubstringWithoutRepeating(const string& s) {
    vector<int> freq(256, 0);
    int left = 0;
    int best = 0;

    for (int right = 0; right < (int)s.size(); right++) {
        unsigned char current = s[right];
        freq[current]++;

        while (freq[current] > 1) {
            freq[(unsigned char)s[left]]--;
            left++;
        }

        best = max(best, right - left + 1);
    }

    return best;
}

int main() {
    string s = "abcabcbb";
    cout << lengthOfLongestSubstringWithoutRepeating(s) << "\n";
    return 0;
}
```

pYTHON IMPLEMENTATION

```
def length_of_longest_substring_without_repeating(s: str) -> int:
    freq = {}
    left = 0
    best = 0

    for right, ch in enumerate(s):
        freq[ch] = freq.get(ch, 0) + 1

        while freq[ch] > 1:
            freq[s[left]] -= 1
            left += 1
```

```
best = max(best, right - left + 1)

return best
```

10. Code Explanation

- `right` expands the window one character at a time.
- `freq` stores how many times each character appears in the current window.
- If the new character becomes duplicated, the `left` pointer moves until the duplicate is removed.
- `best` is updated only after the window is valid.

11. Complexity Analysis

Operation	Time	Space
Sliding scan	$O(n)$	$O(k)$
Add/remove char	$O(1)$ average	Included

`k` is alphabet size or distinct characters in the window.

12. Common Patterns

Pattern	Identify It	Approach	Examples
Longest valid substring	"longest substring with..."	Expand, shrink invalid	Longest Substring Without Repeating
Minimum covering substring	"smallest substring containing..."	Track required counts	Minimum Window Substring
Fixed-size window	"length k substring"	Add right, remove old left	Find All Anagrams

13. Common Mistakes

- Updating answer before restoring validity.
- Confusing substring with subsequence.
- Forgetting to decrement counts when moving `left`.
- Using `if` instead of `while` when multiple removals are needed.
- Off-by-one in window length: `right - left + 1`.

14. Edge Cases

- Empty string.
- All same characters.
- All unique characters.
- Window size larger than string.
- Required characters missing.

- Case-sensitive input.

15. Variations

Variation	Use	Importance
Variable-size window	Longest/shortest valid substring	Very high
Fixed-size window	Anagrams, rolling counts	Very high
At most K	Distinct character constraints	Common
Exactly K	Usually <code>atMost(K) - atMost(K-1)</code>	Important in CP

16. Related Algorithms/Data Structures

- Two pointers: sliding window is a specialized two-pointer technique.
- Character frequency: used to maintain window state.
- Prefix sums: better for numeric fixed range sums.
- Hashing: useful for comparing window content quickly.

17. Practice Problems

Easy

Problem	Platform	Main Idea	Difficulty
Maximum Number of Vowels in a Substring	LeetCode	Fixed-size window	Easy
Longest Nice Substring	LeetCode	Window/brute force hybrid	Easy

Medium

Problem	Platform	Main Idea	Difficulty
Longest Substring Without Repeating Characters	LeetCode	Variable window	Medium
Find All Anagrams in a String	LeetCode	Fixed window frequency	Medium
Longest Repeating Character Replacement	LeetCode	At most replacements	Medium

Hard

Problem	Platform	Main Idea	Difficulty
Minimum Window Substring	LeetCode	Cover required frequency	Hard
Substring with Concatenation of All Words	LeetCode	Word-size sliding window	Hard

18. Interview Explanation

"For substring problems, I maintain a window `[left, right]`. I expand with `right`, update counts, and shrink from `left` until the window satisfies the condition. Since each index moves forward at most once, this

avoids checking all substrings and gives linear time."

19. Revision Notes

- Window length: `right - left + 1`.
- Expand first, then shrink invalid windows.
- Use frequency maps for character constraints.
- `while` is safer than `if` for shrinking.
- Complexity is usually $O(n)$.

20. Final Cheat Sheet

Item	Note
Use when	Contiguous substring with dynamic condition
Main operation	Add right, remove left
Complexity	$O(n)$ time
Key code idea	Maintain invariant before answer update
Edge cases	Empty string, missing required chars, duplicates

Palindrome Checking

1. Overview

A palindrome is a string that reads the same forward and backward, such as `madam` or `racecar`.

2. Intuition

Characters at symmetric positions must match:

- first with last
- second with second-last
- and so on

If every pair matches, the string is a palindrome.

3. When to Use It

Use palindrome checking when problems mention:

- "reads same backward"
- "valid palindrome"
- "can become palindrome"
- "palindromic substring"
- "remove at most one character"

4. When Not to Use It

Avoid simple checking when:

- You need all palindromic substrings; use center expansion, DP, Manacher, or Eertree.
- You need many substring palindrome queries; use hashing or DP.
- You need longest palindromic substring in linear time; use Manacher.

5. Core Concepts

Concept	Meaning	Example
Symmetry	$s[i] == s[n-1-i]$	racecar
Center	Middle of palindrome	e in level
Odd length	One center char	aba
Even length	Two center chars	abba

6. Step-by-Step Algorithm

1. Set `left = 0`, `right = n - 1`.
2. While `left < right`, compare `s[left]` and `s[right]`.
3. If they differ, return `false`.
4. Move both pointers inward.
5. Return `true`.

7. Dry Run

Input: `s = "abba"`

Step	left	right	Compare
1	0	3	<code>a == a</code>
2	1	2	<code>b == b</code>

Final answer: `true`.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

bool isPalindrome(const string& s, int left, int right) {
    while (left < right) {
        if (s[left] != s[right]) return false;
        left++;
        right--;
    }
    return true;
}
```

```

bool validPalindromeAfterAtMostOneDeletion(const string& s) {
    int left = 0, right = (int)s.size() - 1;

    while (left < right) {
        if (s[left] == s[right]) {
            left++;
            right--;
        } else {
            return isPalindrome(s, left + 1, right) ||
                   isPalindrome(s, left, right - 1);
        }
    }

    return true;
}

int main() {
    cout << boolalpha << isPalindrome("abba", 0, 3) << "\n";
    cout << validPalindromeAfterAtMostOneDeletion("abca") << "\n";
    return 0;
}

```

pYTHON IMPLEMENTATION

```

def is_palindrome_range(s: str, left: int, right: int) -> bool:
    while left < right:
        if s[left] != s[right]:
            return False
        left += 1
        right -= 1
    return True

def valid_palindrome_after_at_most_one_deletion(s: str) -> bool:
    left, right = 0, len(s) - 1

    while left < right:
        if s[left] == s[right]:
            left += 1
            right -= 1
        else:
            return (
                is_palindrome_range(s, left + 1, right)
                or is_palindrome_range(s, left, right - 1)
            )

    return True

```

10. Code Explanation

- `isPalindrome` checks a substring range without copying.
- The one-deletion function scans normally until a mismatch.
- At the first mismatch, only two choices can fix it: skip the left character or skip the right character.
- If either remaining range is a palindrome, the answer is true.

11. Complexity Analysis

Operation	Time	Space
Basic check	$O(n)$	$O(1)$
At most one deletion	$O(n)$	$O(1)$

12. Common Patterns

Pattern	Identify It	Approach	Examples
Direct palindrome	"is palindrome"	Two pointers	Valid Palindrome
One deletion	"remove at most one"	Try both skips on mismatch	Valid Palindrome II
Palindromic substring	"longest palindrome substring"	Expand centers or Manacher	Longest Palindromic Substring

13. Common Mistakes

- Copying substrings repeatedly, causing $O(n^2)$ behavior.
- Not handling even-length palindromes.
- For one deletion, skipping only one side.
- Confusing palindromic substring with palindromic subsequence.

14. Edge Cases

- Empty string.
- One character.
- Two equal characters.
- Two different characters.
- All same characters.
- Mismatch near the center.

15. Variations

Variation	Use
Case-insensitive palindrome	Ignore case and punctuation
At most one deletion	Interview favorite
Palindrome queries	Hashing or DP
Longest palindrome	Center expansion or Manacher

16. Related Algorithms/Data Structures

- Two pointers: simplest palindrome check.
- Manacher: longest palindromic substring in $O(n)$.
- Eertree: stores all distinct palindromic substrings online.
- DP: palindromic subsequence and substring tables.

17. Practice Problems

Easy

Problem	Platform	Main Idea	Difficulty
Valid Palindrome	LeetCode	Two pointers	Easy
Palindrome Number	LeetCode	Symmetry idea	Easy

Medium

Problem	Platform	Main Idea	Difficulty
Valid Palindrome II	LeetCode	Skip one mismatch	Medium
Longest Palindromic Substring	LeetCode	Expand around centers	Medium
Palindromic Substrings	LeetCode	Count centers	Medium

Hard

Problem	Platform	Main Idea	Difficulty
Shortest Palindrome	LeetCode	KMP/hash prefix palindrome	Hard
Palindrome Pairs	LeetCode	Trie/hash palindrome checks	Hard

18. Interview Explanation

"A palindrome has matching characters at symmetric positions. I check it with two pointers from both ends and move inward. On mismatch, it is not a palindrome unless the problem allows deletion, in which case I test the two possible skipped ranges."

19. Revision Notes

- Compare $s[l]$ and $s[r]$.
- Move inward after match.
- One deletion: try $(l+1, r)$ and $(l, r-1)$.
- Do not copy substrings.
- Complexity: $O(n)$.

20. Final Cheat Sheet

Item	Note
Use when	Symmetry in string
Main operation	Compare ends
Complexity	$O(n)$ time, $O(1)$ space
Key code idea	Range helper
Edge cases	Empty, one char, even length, allowed deletion

String Reversal

1. Overview

String reversal means producing the characters of a string in opposite order. It is simple but appears inside many larger string problems.

2. Intuition

Swap the first character with the last, the second with the second-last, and continue until the middle.

3. When to Use It

Use it for:

- "reverse string"
- "reverse words"
- "check reverse equality"
- stack-like undoing
- building suffix/prefix relationships

4. When Not to Use It

Avoid it when:

- You only need to iterate backward; no need to allocate reversed string.
- Unicode grapheme clusters matter; byte/char reversal may break characters.
- You need reverse lookup over many strings; consider trie on reversed words.

5. Core Concepts

Concept	Meaning	Why It Matters
In-place reversal	Swap inside same string	$O(1)$ extra space
Copy reversal	Create new reversed string	Original remains unchanged
Word reversal	Reverse word order, not characters	Common interview problem

6. Step-by-Step Algorithm

1. Set `left = 0`, `right = n - 1`.
2. Swap `s[left]` and `s[right]`.
3. Increment `left`, decrement `right`.
4. Stop when `left >= right`.

7. Dry Run

Input: `s = "abcd"`

Step	left	right	String
Initial	0	3	abcd
1	0	3	dbca
2	1	2	dcba

Final answer: `dcba`.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

void reverseInPlace(string& s) {
    int left = 0;
    int right = (int)s.size() - 1;

    while (left < right) {
        swap(s[left], s[right]);
        left++;
        right--;
    }
}

string reversedCopy(string s) {
    reverseInPlace(s);
    return s;
}

int main() {
    string s = "abcd";
    reverseInPlace(s);
    cout << s << "\n";
    cout << reversedCopy("placement") << "\n";
    return 0;
}
```

pYTHON IMPLEMENTATION

```
def reversed_copy(s: str) -> str:
    return s[::-1]

def reverse_list_in_place(chars: list[str]) -> None:
    left, right = 0, len(chars) - 1
    while left < right:
        chars[left], chars[right] = chars[right], chars[left]
        left += 1
        right -= 1
```

10. Code Explanation

- The C++ in-place function takes `string&` so it modifies the original.
- `swap` exchanges symmetric characters.
- `reversedCopy` accepts by value, reverses the copy, and returns it.
- Python strings are immutable, so slicing returns a new reversed string.

11. Complexity Analysis

Operation	Time	Space
In-place reverse	$O(n)$	$O(1)$
Reversed copy	$O(n)$	$O(n)$

12. Common Patterns

Pattern	Identify It	Approach	Examples
Reverse characters	Direct reversal	Two pointers	Reverse String
Reverse words	Word order changes	Trim, split, reverse	Reverse Words in a String
Reverse segments	Every k chars	Process blocks	Reverse String II

13. Common Mistakes

- Returning a reference to a local reversed string.
- Forgetting strings are immutable in Python.
- Reversing characters when the problem asks to reverse words.
- Mishandling multiple spaces in word reversal.

14. Edge Cases

- Empty string.
- One character.
- Even length.
- Odd length.

- Spaces at beginning/end.
- Multiple spaces between words.

15. Variations

Variation	Use
Reverse whole string	Basic operation
Reverse each word	Preserve word order
Reverse word order	Common placement problem
Reverse every k chars	Simulation problem

16. Related Algorithms/Data Structures

- Two pointers: standard implementation.
- Stack: reversing by pushing and popping.
- Deque: useful when building strings from both ends.

17. Practice Problems

Easy

Problem	Platform	Main Idea	Difficulty
Reverse String	LeetCode	Two-pointer swap	Easy
Reverse Vowels of a String	LeetCode	Two pointers with filter	Easy

Medium

Problem	Platform	Main Idea	Difficulty
Reverse Words in a String	LeetCode	Split/trim/reverse	Medium
Reverse String II	LeetCode	Reverse blocks	Medium

Hard

Problem	Platform	Main Idea	Difficulty
Shortest Palindrome	LeetCode	Reverse plus KMP/hash	Hard
Palindrome Pairs	LeetCode	Reversed words + trie/hash	Hard

18. Interview Explanation

"To reverse a string in place, I use two pointers from both ends and swap characters until they meet. This touches each character at most once and uses constant extra space."

19. Revision Notes

- In-place: `swap(s[l++], s[r--])`.
- Copy: `reverse(t.begin(), t.end())`.
- Python: `s[::-1]`.
- Clarify whether reversing characters or words.

20. Final Cheat Sheet

Item	Note
Use when	Need opposite order
Main operation	Swap symmetric chars
Complexity	$O(n)$
Key code idea	<code>while (l < r)</code>
Edge cases	Empty, one char, spaces

Substring Problems

1. Overview

A substring is a contiguous part of a string. Substring problems ask about finding, counting, comparing, or optimizing contiguous ranges.

2. Intuition

Every substring is determined by a start and end index. Brute force tries all pairs, but good solutions exploit structure: sliding windows, hashing, prefix functions, or DP.

3. When to Use It

Recognize substring problems from:

- "contiguous"
- "substring"
- "longest/shortest substring"
- "occurs in string"
- "count substrings"
- "substring equal to pattern"

4. When Not to Use It

Do not use substring techniques for:

- Subsequences, where characters need not be adjacent.
- Rearrangement-only questions; use frequency.
- Full dictionary prefix search; use trie.
- Many suffix-related lexicographic queries; use suffix array/automaton.

5. Core Concepts

Concept	Meaning	Example
Substring	Contiguous range	"abc" in "zabcx"
Subsequence	Ordered but not contiguous	"ac" in "abc"
Start/end indices	Define a substring	<code>s[l..r]</code>
Window state	Info about current substring	Frequency counts

6. Step-by-Step Algorithm

General optimized approach:

1. Identify whether the substring length is fixed or variable.
2. Decide what makes a substring valid.
3. Maintain the validity state using sliding window, hashing, or DP.
4. Update the answer when a candidate substring is valid.
5. Avoid copying substrings unless required for output.

7. Dry Run

Count substrings of length 3 in `s = "abcde"`.

Start	End	Substring
0	2	abc
1	3	bcd
2	4	cde

Count: 3.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

vector<string> allSubstringsOfLengthK(const string& s, int k) {
    vector<string> result;
    if (k <= 0 || k > (int)s.size()) return result;

    for (int start = 0; start + k <= (int)s.size(); start++) {
        result.push_back(s.substr(start, k));
    }

    return result;
}
```

```

int countSubstringsWithAtMostKDistinct(const string& s, int k) {
    if (k < 0) return 0;

    unordered_map<char, int> freq;
    int left = 0;
    int answer = 0;

    for (int right = 0; right < (int)s.size(); right++) {
        freq[s[right]]++;

        while ((int)freq.size() > k) {
            char removeChar = s[left++];
            freq[removeChar]--;
            if (freq[removeChar] == 0) freq.erase(removeChar);
        }

        answer += right - left + 1;
    }

    return answer;
}

int main() {
    for (const string& sub : allSubstringsOfLengthK("abcde", 3)) {
        cout << sub << "\n";
    }
    cout << countSubstringsWithAtMostKDistinct("pqpqs", 2) << "\n";
    return 0;
}

```

pYTHON IMPLEMENTATION

```

from collections import defaultdict

def all_substrings_of_length_k(s: str, k: int) -> list[str]:
    if k <= 0 or k > len(s):
        return []
    return [s[i:i + k] for i in range(len(s) - k + 1)]

def count_substrings_with_at_most_k_distinct(s: str, k: int) -> int:
    if k < 0:
        return 0

    freq = defaultdict(int)
    left = 0
    answer = 0

    for right, ch in enumerate(s):
        freq[ch] += 1

```

```

while len(freq) > k:
    remove = s[left]
    left += 1
    freq[remove] -= 1
    if freq[remove] == 0:
        del freq[remove]
    answer += right - left + 1

return answer

```

10. Code Explanation

- `allSubstringsOfLengthK` shows careful bounds: last start is $n-k$.
- `countSubstringsWithAtMostKDistinct` uses the fact that if `s[left..right]` is valid, every suffix ending at `right` inside that window is also valid.
- The answer adds `right - left + 1` valid substrings ending at `right`.

11. Complexity Analysis

Problem Type	Time	Space
Generate fixed-length substrings	$O((n-k+1)k)$	Output size
Count with sliding window	$O(n)$	$O(k)$
Brute force all substrings	$O(n^2)$ ranges, $O(n^3)$ if copying badly	Varies

12. Common Patterns

Pattern	Identify It	Approach	Examples
Longest valid substring	Max length contiguous	Sliding window	Longest Substring Without Repeating
Count valid substrings	Number of ranges	Sliding window/DP	Count Substrings With K Distinct
Pattern occurrence	Find pattern in text	KMP/Z/Rabin-Karp	Implement strStr

13. Common Mistakes

- Confusing substring with subsequence.
- Calling `substr` inside nested loops unnecessarily.
- Off-by-one in `start + k <= n`.
- Using `int` for huge counts; use `long long` if count can be $O(n^2)$.

14. Edge Cases

- Empty string.
- $k = 0$.

- $k > n$.
- All characters same.
- All characters distinct.
- Large n , where $O(n^2)$ is impossible.

15. Variations

Variation	Use
Fixed-length substrings	Anagrams, hashes
Variable-length substrings	Longest/shortest conditions
Palindromic substrings	Center expansion/Manacher
Distinct substrings	Suffix array/automaton

16. Related Algorithms/Data Structures

- Sliding window: best for contiguous constraints.
- KMP/Z/Rabin-Karp: pattern occurrence.
- Suffix array/automaton: distinct substring and suffix queries.
- Manacher/Eertree: palindromic substring problems.

17. Practice Problems

Easy

Problem	Platform	Main Idea	Difficulty
Implement strStr	LeetCode	Substring search	Easy
Substrings of Size Three with Distinct Characters	LeetCode	Fixed window	Easy

Medium

Problem	Platform	Main Idea	Difficulty
Longest Substring Without Repeating Characters	LeetCode	Sliding window	Medium
Number of Substrings Containing All Three Characters	LeetCode	Count valid windows	Medium
Palindromic Substrings	LeetCode	Center expansion	Medium

Hard

Problem	Platform	Main Idea	Difficulty
Distinct Substrings	CSES	Suffix array/automaton	Hard
Minimum Window Substring	LeetCode	Window coverage	Hard

18. Interview Explanation

"For substring problems, I first confirm the range must be contiguous. Then I choose a technique based on the condition: sliding window for maintainable constraints, KMP/Z/Rabin-Karp for pattern matching, and suffix structures for many suffix or distinct-substring queries."

19. Revision Notes

- Substring is contiguous.
- $n(n+1)/2$ total substrings.
- Avoid copying in loops.
- Use `long long` for counts.
- Match technique to condition.

20. Final Cheat Sheet

Item	Note
Use when	Contiguous ranges
Main operation	Maintain or compare <code>s[l..r]</code>
Complexity	From $O(n)$ to $O(n^2)$ depending on method
Key code idea	Avoid unnecessary <code>substr</code>
Edge cases	Empty, $k > n$, huge counts

Anagram Problems

1. Overview

Anagrams are strings that contain the same characters with the same frequencies, possibly in different order.

2. Intuition

If order does not matter, only counts matter. `listen` and `silent` both contain the same letters, so their frequency signatures match.

3. When to Use It

Use anagram logic when the problem says:

- "rearrange"
- "permutation"
- "same characters"
- "group anagrams"
- "find anagrams in a string"
- "check if one string's permutation is substring"

4. When Not to Use It

Avoid frequency-only anagram logic when:

- Order matters.
- Characters can be used with transformations.
- You need approximate matching.
- You need many prefix matches; use trie.

5. Core Concepts

Concept	Meaning	Example
Frequency signature	Count vector representing string	a1b1t1
Sorted signature	Sorted string as key	eat -> aet
Window anagram	Current substring frequency equals pattern	Find all anagrams

6. Step-by-Step Algorithm

For checking two strings:

1. If lengths differ, return false.
2. Count characters in the first string.
3. Subtract characters from the second string.
4. If all counts are zero, return true.

For finding anagrams in text:

1. Count pattern frequency.
2. Slide a window of pattern length over text.
3. Compare window frequency to pattern frequency.

7. Dry Run

Check `a = "listen"`, `b = "silent"`.

Action	Result
Count listen	l:1 i:1 s:1 t:1 e:1 n:1
Subtract silent	all counts become 0

Final answer: anagrams.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

bool isAnagram(const string& a, const string& b) {
    if (a.size() != b.size()) return false;
```

```

vector<int> freq(26, 0);
for (char ch : a) freq[ch - 'a']++;
for (char ch : b) freq[ch - 'a']--;

for (int count : freq) {
    if (count != 0) return false;
}
return true;
}

vector<int> findAnagrams(const string& text, const string& pattern) {
    vector<int> result;
    int n = text.size(), m = pattern.size();
    if (m > n) return result;

    vector<int> need(26, 0), window(26, 0);
    for (char ch : pattern) need[ch - 'a']++;

    for (int right = 0; right < n; right++) {
        window[text[right] - 'a']++;

        if (right >= m) {
            window[text[right - m] - 'a']--;
        }

        if (right >= m - 1 && window == need) {
            result.push_back(right - m + 1);
        }
    }

    return result;
}

int main() {
    cout << boolalpha << isAnagram("listen", "silent") << "\n";
    vector<int> positions = findAnagrams("cbaebabacd", "abc");
    for (int pos : positions) cout << pos << " ";
    cout << "\n";
    return 0;
}

```

pYTHON IMPLEMENTATION

```

from collections import Counter

def is_anagram(a: str, b: str) -> bool:
    return len(a) == len(b) and Counter(a) == Counter(b)

```

```
def find_anagrams(text: str, pattern: str) -> list[int]:
    n, m = len(text), len(pattern)
    if m > n:
        return []

    need = Counter(pattern)
    window = Counter()
    answer = []

    for right, ch in enumerate(text):
        window[ch] += 1

        if right >= m:
            old = text[right - m]
            window[old] -= 1
            if window[old] == 0:
                del window[old]

        if right >= m - 1 and window == need:
            answer.append(right - m + 1)

    return answer
```

10. Code Explanation

- `isAnagram` uses a 26-size frequency array for lowercase strings.
- `findAnagrams` maintains a fixed-size window equal to pattern length.
- When `right >= m`, the character leaving the window is removed.
- Equality of two 26-size arrays is $O(26)$, effectively constant.

11. Complexity Analysis

Operation	Time	Space
Check two anagrams	$O(n)$	$O(1)$
Find all anagrams	$O(n * \text{alphabet})$ or $O(n)$ for fixed alphabet	$O(1)$
Group anagrams by sorted key	$O(n * L \log L)$	$O(nL)$

12. Common Patterns

Pattern	Identify It	Approach	Examples
Pair check	Two strings	Frequency compare	Valid Anagram
Grouping	List of strings	Signature as hash key	Group Anagrams
Anagram windows	Pattern permutations in text	Fixed sliding window	Find All Anagrams

13. Common Mistakes

- Forgetting length comparison.
- Using sorted strings when frequency signature is faster for fixed alphabet.
- Not deleting zero-count keys in Python `Counter` window comparisons.
- Assuming lowercase without checking constraints.

14. Edge Cases

- Empty strings.
- Different lengths.
- Repeated characters.
- Pattern longer than text.
- Uppercase/lowercase.
- Non-letter characters.

15. Variations

Variation	Use
Sorted key	Simple grouping
Frequency tuple key	Faster for fixed alphabet
Sliding-window anagram	Pattern occurrence by permutation
Minimum changes to anagram	Compare count differences

16. Related Algorithms/Data Structures

- Character frequency: core tool.
- Sliding window: find anagrams inside a longer string.
- Hash map: group by signature.
- Trie: useful for word dictionaries, not direct anagram equality.

17. Practice Problems

Easy

Problem	Platform	Main Idea	Difficulty
Valid Anagram	LeetCode	Frequency equality	Easy
Check If Two String Arrays are Equivalent	LeetCode	Compare built strings	Easy

Medium

Problem	Platform	Main Idea	Difficulty
Group Anagrams	LeetCode	Signature key	Medium
Find All Anagrams in a String	LeetCode	Sliding window	Medium
Permutation in String	LeetCode	Fixed window	Medium

Hard

Problem	Platform	Main Idea	Difficulty
Minimum Window Substring	LeetCode	Frequency cover	Hard
Valid Anagram-related OA variants	HackerRank/GFG	Count differences	Hard in constraints

18. Interview Explanation

"Anagrams have the same character frequencies. For lowercase strings, I use a 26-size array and compare counts in linear time. For finding anagrams inside a text, I maintain a fixed-size sliding window and compare its frequency with the pattern frequency."

19. Revision Notes

- Anagram = same multiset of characters.
- Check length first.
- Lowercase: 26-count vector.
- Window size equals pattern length.
- Trap: zero-count map keys can break equality.

20. Final Cheat Sheet

Item	Note
Use when	Rearrangement/permutation
Main operation	Compare frequency signatures
Complexity	$O(n)$ for fixed alphabet
Key code idea	Add pattern, slide text window
Edge cases	Repeats, length mismatch, pattern > text

Longest Common Prefix

1. Overview

The longest common prefix (LCP) of a set of strings is the longest starting substring shared by all strings.

2. Intuition

Start with the first string as the possible answer. Compare it with every other string and shrink it whenever a mismatch appears.

3. When to Use It

Use it when:

- "common prefix"
- "prefix shared by all strings"
- autocomplete grouping
- lexicographic neighbor comparison
- suffix array LCP computations

4. When Not to Use It

Avoid simple LCP when:

- You need many prefix queries dynamically; use trie.
- You need LCP of suffixes many times; use suffix array + LCP array/RMQ.
- You need longest common substring, not prefix.

5. Core Concepts

Concept	Meaning	Example
Prefix	Starts at index 0	<code>fl</code> in <code>flower</code>
Common prefix	Prefix shared by strings	<code>fl</code> for <code>flower</code> , <code>flow</code>
Horizontal scan	Compare answer with each string	Simple approach
Sorting trick	LCP of first and last sorted strings	Efficient and elegant

6. Step-by-Step Algorithm

Horizontal scanning:

1. If the list is empty, return empty string.
2. Set `prefix = first string`.
3. For each next string, compare characters from the start.
4. Shrink `prefix` to the matched part.
5. If prefix becomes empty, stop.

7. Dry Run

Input: `["flower", "flow", "flight"]`

Compare	Current Prefix	Matched Prefix
start	flower	flower
with flow	flower	flow
with flight	flow	fl

Final answer: `fl`.

8. C++ Implementation

```

#include <bits/stdc++.h>
using namespace std;

string longestCommonPrefix(vector<string>& words) {
    if (words.empty()) return "";

    string prefix = words[0];

    for (int i = 1; i < (int)words.size(); i++) {
        int j = 0;
        while (j < (int)prefix.size() &&
                j < (int)words[i].size() &&
                prefix[j] == words[i][j]) {
            j++;
        }

        prefix = prefix.substr(0, j);
        if (prefix.empty()) break;
    }

    return prefix;
}

string longestCommonPrefixBySorting(vector<string> words) {
    if (words.empty()) return "";
    sort(words.begin(), words.end());

    const string& first = words.front();
    const string& last = words.back();
    int i = 0;
    while (i < (int)first.size() &&
            i < (int)last.size() &&
            first[i] == last[i]) {
        i++;
    }
    return first.substr(0, i);
}

int main() {
    vector<string> words = {"flower", "flow", "flight"};
    cout << longestCommonPrefix(words) << "\n";
    cout << longestCommonPrefixBySorting(words) << "\n";
    return 0;
}

```

pYTHON IMPLEMENTATION

```

def longest_common_prefix(words: list[str]) -> str:
    if not words:
        return ""

```

```

prefix = words[0]
for word in words[1:]:
    i = 0
    while i < len(prefix) and i < len(word) and prefix[i] == word[i]:
        i += 1
    prefix = prefix[:i]
    if not prefix:
        break

return prefix

def longest_common_prefix_by_sorting(words: list[str]) -> str:
    if not words:
        return ""
    words = sorted(words)
    first, last = words[0], words[-1]
    i = 0
    while i < len(first) and i < len(last) and first[i] == last[i]:
        i += 1
    return first[:i]

```

10. Code Explanation

- The horizontal scan keeps a current prefix candidate.
- For each word, `j` advances while characters match.
- `prefix.substr(0, j)` shrinks the candidate.
- Sorting works because the most different strings after sorting are the first and last; their shared prefix is shared by all.

11. Complexity Analysis

Method	Time	Space
Horizontal scan	$O(\text{total characters})$	$O(1)$ extra
Sorting method	$O(n \log n * L)$	$O(1)$ or sorting space
Trie method	$O(\text{total characters})$	$O(\text{total characters})$

12. Common Patterns

Pattern	Identify It	Approach	Examples
Common prefix of list	Direct phrase	Horizontal scan	LeetCode Longest Common Prefix
Prefix dictionary	Many insert/query words	Trie	Autocomplete
Suffix LCP	LCP between suffixes	Suffix array LCP	CSES String Algorithms

13. Common Mistakes

- Not handling empty vector.
- Accessing beyond the shortest string.
- Confusing common prefix with common substring.
- Sorting and forgetting it changes original order if order matters later.

14. Edge Cases

- Empty list.
- One word.
- Empty string inside list.
- No common prefix.
- All words same.
- One word is prefix of another.

15. Variations

Variation	Use
Horizontal scanning	Simple interview solution
Sorting first/last	Short elegant solution
Trie-based LCP	Many words with shared prefix structure
Suffix array LCP	String suffix queries

16. Related Algorithms/Data Structures

- Trie: best for repeated prefix queries.
- Suffix array: uses LCP between suffixes.
- Z algorithm: computes prefix matches for one string.
- KMP prefix function: prefix-suffix relationships.

17. Practice Problems

Easy

Problem	Platform	Main Idea	Difficulty
Longest Common Prefix	LeetCode	Horizontal scan	Easy
Find the Index of the First Occurrence	LeetCode	Prefix matching concept	Easy

Medium

Problem	Platform	Main Idea	Difficulty
Search Suggestions System	LeetCode	Prefix grouping	Medium
Implement Trie	LeetCode	Prefix tree	Medium

Hard

Problem	Platform	Main Idea	Difficulty
Longest Duplicate Substring	LeetCode	Suffix/hash LCP style	Hard
Repeated Substring Queries	Codeforces	Hash/LCP ideas	Hard

18. Interview Explanation

"I keep the first string as the current prefix and compare it with each remaining word. Whenever a mismatch occurs, I shrink the prefix. Since a valid answer must be a prefix of every word, repeated shrinking gives the longest common prefix."

19. Revision Notes

- Empty list returns "".
- Stop early if prefix becomes empty.
- Horizontal scan is $O(\text{total chars})$.
- Sorting trick compares first and last.
- Not the same as longest common substring.

20. Final Cheat Sheet

Item	Note
Use when	Shared beginning of strings
Main operation	Compare from index 0
Complexity	$O(\text{total chars})$
Key code idea	Shrink current prefix
Edge cases	Empty input, no prefix, one word

Basic Hashing

1. Overview

Hashing maps data to a compact value. In string problems, hashing is used to store seen strings, count objects, group equivalent strings, or compare substrings efficiently.

2. Intuition

A hash is like a label for a value. Instead of comparing whole strings every time, store or compare their labels. For exact correctness in interviews, use maps/sets directly; for substring hashing, understand collision risk.

3. When to Use It

Use hashing when:

- "count distinct"

- "seen before"
- "group by signature"
- "detect duplicates"
- "fast lookup"
- "compare substrings many times"

4. When Not to Use It

Avoid hashing when:

- Ordered traversal is required; use map, sort, trie, or suffix array.
- Collision risk is unacceptable and no verification is done.
- A small fixed array is simpler, such as lowercase character frequency.
- You need prefix search; use trie.

5. Core Concepts

Concept	Meaning	Why It Matters
Hash table	Stores key-value pairs	Fast average lookup
Set	Stores unique keys	Distinct detection
Map	Key to value/count	Frequencies
Collision	Different keys same hash	Can affect custom rolling hash

6. Step-by-Step Algorithm

For common hash map counting:

1. Create `unordered_map<string, int>`.
2. Iterate over items.
3. Increment count for each key.
4. Query counts or distinct keys as needed.

7. Dry Run

Words: `["eat", "tea", "eat"]`

Word	Map State
eat	eat:1
tea	eat:1, tea:1
eat	eat:2, tea:1

Distinct words: 2.

8. C++ Implementation

```

#include <bits/stdc++.h>
using namespace std;

unordered_map<string, int> countWords(const vector<string>& words) {
    unordered_map<string, int> freq;
    for (const string& word : words) {
        freq[word]++;
    }
    return freq;
}

vector<vector<string>> groupAnagrams(vector<string>& words) {
    unordered_map<string, vector<string>> groups;

    for (string word : words) {
        string key = word;
        sort(key.begin(), key.end());
        groups[key].push_back(word);
    }

    vector<vector<string>> result;
    for (auto& [key, group] : groups) {
        result.push_back(group);
    }
    return result;
}

int main() {
    vector<string> words = {"eat", "tea", "tan", "ate", "nat", "bat"};
    auto freq = countWords(words);
    cout << freq["eat"] << "\n";

    auto groups = groupAnagrams(words);
    cout << groups.size() << "\n";
    return 0;
}

```

pYTHON IMPLEMENTATION

```

from collections import Counter, defaultdict

def count_words(words: list[str]) -> dict[str, int]:
    return dict(Counter(words))

def group_anagrams(words: list[str]) -> list[list[str]]:
    groups = defaultdict(list)
    for word in words:
        key = "".join(sorted(word))

```

```
groups[key].append(word)
return list(groups.values())
```

10. Code Explanation

- `unordered_map` gives average $O(1)$ insertion and lookup.
- `countWords` uses the string itself as the key.
- `groupAnagrams` uses the sorted string as a canonical key.
- Python `Counter` and `defaultdict` are standard tools for the same pattern.

11. Complexity Analysis

Operation	Time	Space
Insert/find in hash table	$O(1)$ average	$O(n)$ total
Count words	$O(\text{total chars})$ average	$O(\text{unique words})$
Group anagrams with sorting	$O(n * L \log L)$	$O(nL)$

12. Common Patterns

Pattern	Identify It	Approach	Examples
Seen before	Duplicate detection	<code>unordered_set</code>	Contains Duplicate
Counting	Frequency of keys	<code>unordered_map</code>	Word Pattern
Group by signature	Equivalent items	Hash key	Group Anagrams

13. Common Mistakes

- Forgetting `unordered_map` has worst-case $O(n)$ operations.
- Using custom rolling hash without considering collisions.
- Using hash map when a 26-array is simpler.
- Relying on iteration order of `unordered_map`.

14. Edge Cases

- Empty list.
- Empty strings as keys.
- Very long strings.
- Many duplicate keys.
- Case-sensitive keys.
- Hash collision-sensitive problems.

15. Variations

Variation	Use
<code>unordered_set</code>	Membership/distinct
<code>unordered_map</code>	Counts and grouping
Ordered <code>map</code>	Sorted key traversal
Rolling hash	Substring comparison

16. Related Algorithms/Data Structures

- Character frequency: fixed-array hashing by character.
- Rolling hash/Rabin-Karp: hash substrings.
- Trie: better for prefix-based lookup.
- Suffix array: better for ordered suffix queries.

17. Practice Problems

Easy

Problem	Platform	Main Idea	Difficulty
Contains Duplicate	LeetCode	Hash set	Easy
Isomorphic Strings	LeetCode	Character mapping	Easy

Medium

Problem	Platform	Main Idea	Difficulty
Group Anagrams	LeetCode	Hash signature	Medium
Top K Frequent Words	LeetCode	Hash + heap/sort	Medium
Longest Consecutive Sequence	LeetCode	Hash set	Medium

Hard

Problem	Platform	Main Idea	Difficulty
Longest Duplicate Substring	LeetCode	Rolling hash + binary search	Hard
Word Ladder	LeetCode	Hash patterns + BFS	Hard

18. Interview Explanation

"Hashing gives fast average lookup for strings or signatures. I use sets for membership, maps for counts, and canonical keys for grouping. If I use rolling hash for substrings, I mention collision handling or use double hashing."

19. Revision Notes

- `unordered_map` average $O(1)$.
- Use `unordered_set` for existence.
- Use canonical key for grouping.
- Do not rely on unordered iteration order.
- For substring hashes, collisions exist.

20. Final Cheat Sheet

Item	Note
Use when	Fast lookup/count/group
Main operation	Insert/find key
Complexity	Average $O(1)$ per operation
Key code idea	<code>mp[key]++</code>
Edge cases	Empty keys, duplicates, collisions

KMP Algorithm

1. Overview

KMP (Knuth-Morris-Pratt) finds occurrences of a pattern inside a text in linear time using a prefix function.

2. Intuition

When a mismatch happens, do not restart from scratch. KMP remembers the longest prefix of the pattern that is also a suffix of what has matched so far. This tells how far the pattern can shift safely.

Analogy: If you matched `abab` and fail at the next character, you already know the suffix `ab` can still be useful because it is also a prefix.

3. When to Use It

Use KMP when:

- "find pattern in text"
- "string matching in $O(n)$ "
- "prefix equals suffix"
- "shortest palindrome"
- "repeated substring pattern"
- multiple overlapping matches matter

4. When Not to Use It

Avoid KMP when:

- Pattern and text are tiny; built-in find is acceptable in interviews only if allowed.
- You need many dictionary patterns; use Aho-Corasick.

- You need approximate matching; KMP is exact.
- You need lexicographic suffix queries; use suffix array.

5. Core Concepts

Concept	Meaning	Example
Prefix function <code>pi[i]</code>	Length of longest proper prefix ending at <code>i</code> that is also suffix	<code>abab</code> has <code>pi[3]=2</code>
Proper prefix	Prefix not equal to whole string	<code>ab</code> for <code>abab</code>
Fallback	Move matched length to <code>pi[j-1]</code>	Avoids restarting
Overlap	Matches can share characters	Pattern <code>aa</code> in <code>aaaa</code>

6. Step-by-Step Algorithm

Build prefix function:

1. Set `pi[0] = 0`.
2. For each index `i`, start with `j = pi[i - 1]`.
3. While mismatch and `j > 0`, set `j = pi[j - 1]`.
4. If characters match, increment `j`.
5. Store `pi[i] = j`.

Search:

1. Build `pi` for pattern.
2. Scan text while maintaining matched length `j`.
3. On mismatch, fallback using `pi`.
4. On match, increment `j`.
5. If `j == pattern length`, record occurrence and fallback for overlaps.

7. Dry Run

Pattern: `ababaca`

i	char	pi[i]	Reason
0	a	0	no proper prefix
1	b	0	no match
2	a	1	a
3	b	2	ab
4	a	3	aba
5	c	0	fallback fails
6	a	1	a

`pi = [0,0,1,2,3,0,1]`

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

vector<int> prefixFunction(const string& s) {
    int n = s.size();
    vector<int> pi(n, 0);

    for (int i = 1; i < n; i++) {
        int j = pi[i - 1];

        while (j > 0 && s[i] != s[j]) {
            j = pi[j - 1];
        }

        if (s[i] == s[j]) {
            j++;
        }

        pi[i] = j;
    }

    return pi;
}

vector<int> kmpSearch(const string& text, const string& pattern) {
    vector<int> positions;
    if (pattern.empty()) return positions;

    vector<int> pi = prefixFunction(pattern);
    int j = 0;

    for (int i = 0; i < (int)text.size(); i++) {
        while (j > 0 && text[i] != pattern[j]) {
            j = pi[j - 1];
        }

        if (text[i] == pattern[j]) {
            j++;
        }

        if (j == (int)pattern.size()) {
            positions.push_back(i - (int)pattern.size() + 1);
            j = pi[j - 1];
        }
    }

    return positions;
}

int main() {
```

```

string text = "ababcbcabababd";
string pattern = "ababd";
vector<int> positions = kmpSearch(text, pattern);
for (int pos : positions) cout << pos << " ";
cout << "\n";
return 0;
}

```

pYTHON IMPLEMENTATION

```

def prefix_function(s: str) -> list[int]:
    pi = [0] * len(s)
    for i in range(1, len(s)):
        j = pi[i - 1]
        while j > 0 and s[i] != s[j]:
            j = pi[j - 1]
        if s[i] == s[j]:
            j += 1
        pi[i] = j
    return pi

def kmp_search(text: str, pattern: str) -> list[int]:
    if not pattern:
        return []

    pi = prefix_function(pattern)
    positions = []
    j = 0

    for i, ch in enumerate(text):
        while j > 0 and ch != pattern[j]:
            j = pi[j - 1]
        if ch == pattern[j]:
            j += 1
        if j == len(pattern):
            positions.append(i - len(pattern) + 1)
            j = pi[j - 1]

    return positions

```

10. Code Explanation

- `prefixFunction` precomputes fallback lengths for every pattern prefix.
- `j` represents how many pattern characters are currently matched.
- On mismatch, `j = pi[j - 1]` preserves the longest useful overlap.
- After a full match, fallback enables overlapping matches.

11. Complexity Analysis

Phase	Time	Space
Prefix preprocessing	$O(m)$	$O(m)$
Search	$O(n)$	$O(1)$ extra
Overall	$O(n + m)$	$O(m)$

12. Common Patterns

Pattern	Identify It	Approach	Examples
Find pattern	Exact occurrence	KMP search	Implement strStr
Prefix-suffix	Border of string	Prefix function	Longest Happy Prefix
Repetition	Built from repeated block	Use $pi[n-1]$	Repeated Substring Pattern

13. Common Mistakes

- Using $pi[j]$ instead of $pi[j - 1]$ for fallback.
- Forgetting overlapping matches.
- Accessing $pattern[j]$ when pattern is empty.
- Confusing prefix function with Z array.

14. Edge Cases

- Empty pattern.
- Pattern longer than text.
- Pattern equals text.
- Repeated characters like `aaaa`.
- Overlapping matches.
- No match.

15. Variations

Variation	Use
Concatenate $pattern\#text$	Find matches via prefix values
Prefix-function search	Standard KMP
Automaton KMP	Many transitions, CP use
Shortest palindrome with KMP	Prefix palindrome detection

16. Related Algorithms/Data Structures

- Z algorithm: also computes prefix matches, often simpler for pattern search.
- Rabin-Karp: hashing-based string matching.
- Aho-Corasick: KMP-like failure links for multiple patterns.
- Suffix array: heavier tool for many substring queries.

17. Practice Problems

Easy

Problem	Platform	Main Idea	Difficulty
Find the Index of the First Occurrence	LeetCode	Pattern search	Easy
Repeated Substring Pattern	LeetCode	Prefix function	Easy

Medium

Problem	Platform	Main Idea	Difficulty
Longest Happy Prefix	LeetCode	Border via KMP	Medium
String Matching in an Array	LeetCode	Pattern search	Medium
CSES Finding Patterns	CSES	KMP or Z	Medium

Hard

Problem	Platform	Main Idea	Difficulty
Shortest Palindrome	LeetCode	KMP on <code>s#rev</code>	Hard
Password	Codeforces	Prefix-suffix occurrence	Hard

18. Interview Explanation

"KMP avoids rechecking characters after a mismatch. It precomputes, for every pattern prefix, the longest proper prefix that is also a suffix. During search, if a mismatch occurs after matching `j` characters, I jump to `pi[j-1]` instead of restarting."

19. Revision Notes

- `pi[i]` = longest border ending at `i`.
- Fallback: `j = pi[j - 1]`.
- Search is $O(n + m)$.
- Handles overlapping matches.
- Trap: empty pattern and repeated chars.

20. Final Cheat Sheet

Item	Note
Use when	Exact single-pattern matching
Main operation	Prefix fallback
Complexity	$O(n + m)$
Key code idea	<code>while (j > 0 && mismatch) j = pi[j-1]</code>

Item	Note
Edge cases	Empty pattern, overlaps, repeated chars

Z Algorithm

1. Overview

The Z algorithm computes $z[i]$, the length of the longest substring starting at i that matches the prefix of the string.

2. Intuition

At every index, ask: "How many characters from here match the beginning of the string?" The algorithm reuses a known matching segment $[l, r]$ to avoid repeated comparisons.

3. When to Use It

Use Z algorithm for:

- pattern matching
- prefix matches at every position
- string border problems
- finding occurrences using `pattern + '#' + text`
- problems asking "longest prefix starting at each index"

4. When Not to Use It

Avoid it when:

- You need suffix ordering; use suffix array.
- You need multiple patterns; use Aho-Corasick.
- You only need character counts; use frequency.
- You are more comfortable with KMP and prefix function is directly needed.

5. Core Concepts

Concept	Meaning	Example
$z[i]$	Prefix match length from index i	$z[2]=3$ means $s[2..4] == s[0..2]$
Z-box	Segment $[l, r]$ matching prefix	Reuse known comparisons
Separator	Character not in pattern/text	Prevents false cross-match

6. Step-by-Step Algorithm

1. Initialize $l = r = 0$.
2. For each i from 1 to $n - 1$:
3. If $i \leq r$, initialize $z[i] = \min(r - i + 1, z[i - l])$.

4. Extend `z[i]` by direct comparisons.
5. If the new match goes beyond `r`, update `[l, r]`.

7. Dry Run

String: `aaaaa`

i	Initial z[i]	Extended z[i]
1	0	4
2	3	3
3	2	2
4	1	1

`z = [0,4,3,2,1]`

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

vector<int> zFunction(const string& s) {
    int n = s.size();
    vector<int> z(n, 0);
    int left = 0, right = 0;

    for (int i = 1; i < n; i++) {
        if (i <= right) {
            z[i] = min(right - i + 1, z[i - left]);
        }

        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) {
            z[i]++;
        }

        if (i + z[i] - 1 > right) {
            left = i;
            right = i + z[i] - 1;
        }
    }

    return z;
}

vector<int> zSearch(const string& text, const string& pattern) {
    string combined = pattern + "#" + text;
    vector<int> z = zFunction(combined);
    vector<int> positions;
    int m = pattern.size();

```

```

    for (int i = m + 1; i < (int)combined.size(); i++) {
        if (z[i] == m) {
            positions.push_back(i - m - 1);
        }
    }

    return positions;
}

int main() {
    vector<int> positions = zSearch("ababcbabd", "ab");
    for (int pos : positions) cout << pos << " ";
    cout << "\n";
    return 0;
}

```

pYTHON IMPLEMENTATION

```

def z_function(s: str) -> list[int]:
    z = [0] * len(s)
    left = right = 0

    for i in range(1, len(s)):
        if i <= right:
            z[i] = min(right - i + 1, z[i - left])

            while i + z[i] < len(s) and s[z[i]] == s[i + z[i]]:
                z[i] += 1

            if i + z[i] - 1 > right:
                left = i
                right = i + z[i] - 1

    return z

def z_search(text: str, pattern: str) -> list[int]:
    combined = pattern + "#" + text
    z = z_function(combined)
    m = len(pattern)
    return [i - m - 1 for i in range(m + 1, len(combined)) if z[i] == m]

```

10. Code Explanation

- `[left, right]` stores the farthest segment known to match the prefix.
- If `i` lies inside the Z-box, some answer can be copied from `z[i-left]`.
- The `while` loop extends beyond known information.
- Pattern search checks positions where the prefix match length equals the pattern length.

11. Complexity Analysis

Phase	Time	Space
Z array	$O(n)$	$O(n)$
Pattern search	$O(n + m)$	$O(n + m)$

12. Common Patterns

Pattern	Identify It	Approach	Examples
Pattern search	Find all occurrences	<code>pattern#text</code>	CSES Finding Patterns
Prefix at each index	Longest prefix match	Z array directly	String border tasks
Repetition	Prefix repeats	Analyze Z values	Codeforces string tasks

13. Common Mistakes

- Choosing a separator that appears in input.
- Forgetting `z[0]` is usually set to 0.
- Off-by-one when converting combined index to text index.
- Confusing `z[i]` with prefix function `pi[i]`.

14. Edge Cases

- Empty pattern.
- Pattern longer than text.
- All same characters.
- No match.
- Overlapping matches.
- Separator conflict.

15. Variations

Variation	Use
Standard Z	Prefix matches
Z search	Exact pattern matching
Reverse Z	Suffix/prefix relation
Z with sentinel	Combine strings safely

16. Related Algorithms/Data Structures

- KMP: prefix function alternative.
- Rolling hash: probabilistic substring comparison.
- Suffix array: more powerful suffix queries.
- Aho-Corasick: many patterns.

17. Practice Problems

Easy

Problem	Platform	Main Idea	Difficulty
Implement strStr	LeetCode	Z search	Easy
Repeated Substring Pattern	LeetCode	Prefix repetition	Easy

Medium

Problem	Platform	Main Idea	Difficulty
CSES Finding Borders	CSES	Prefix/suffix via Z	Medium
CSES Finding Patterns	CSES	Z or KMP search	Medium
Longest Happy Prefix	LeetCode	Prefix-suffix	Medium

Hard

Problem	Platform	Main Idea	Difficulty
Codeforces Password	Codeforces	Border occurrence	Hard
String compression variants	Codeforces	Z repetition	Hard

18. Interview Explanation

"The Z array tells how long the prefix matches from every index. The algorithm keeps a matching segment and reuses it, so each character is compared only a constant number of times. For pattern search, I run Z on `pattern + separator + text` and look for Z values equal to pattern length."

19. Revision Notes

- $z[i]$ = LCP of s and $s[i..]$.
- Maintain $[l, r]$ Z-box.
- Pattern index conversion: $i - m - 1$.
- Use safe separator.
- Complexity: $O(n)$.

20. Final Cheat Sheet

Item	Note
Use when	Prefix match at each index
Main operation	Maintain Z-box
Complexity	$O(n)$
Key code idea	Reuse $z[i-1]$ inside $[l, r]$

Item	Note
Edge cases	Separator, overlaps, empty pattern

Rabin-Karp

1. Overview

Rabin-Karp is a string matching algorithm that uses hashing to compare a pattern with text windows efficiently.

2. Intuition

Instead of comparing every character for every window, compute a numeric fingerprint for the pattern and each text window. If fingerprints differ, the strings are definitely different. If they match, verify or rely on strong hashing.

3. When to Use It

Use Rabin-Karp when:

- You need many substring comparisons.
- Pattern length is fixed.
- You need duplicate substring detection.
- You combine binary search with substring existence.
- Multiple patterns of same length are involved.

4. When Not to Use It

Avoid it when:

- Deterministic exact matching is required and collision risk is not acceptable.
- KMP/Z solves the single-pattern problem more simply.
- Mod arithmetic may be too bug-prone for the situation.
- You need many patterns of different lengths; use Aho-Corasick.

5. Core Concepts

Concept	Meaning	Why It Matters
Polynomial hash	Treat string like base-power number	Enables rolling
Rolling update	Remove left char, add right char	Fast window hash
Modulo	Keeps numbers bounded	Avoids overflow
Collision	Same hash for different strings	Must consider verification/double hash

6. Step-by-Step Algorithm

1. Choose base and modulo.
2. Compute hash of the pattern.
3. Compute hash of the first text window.
4. Slide the window one character at a time.
5. If hash equals pattern hash, verify characters or use double hash.
6. Record matching positions.

7. Dry Run

Text: **ababc**, Pattern: **ab**

Window	Hash Match?	Result
ab	yes	record 0
ba	no	skip
ab	yes	record 2
bc	no	skip

Final positions: **[0, 2]**.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

vector<int> rabinKarpSearch(const string& text, const string& pattern) {
    const long long MOD = 1000000007LL;
    const long long BASE = 911382323LL;

    int n = text.size(), m = pattern.size();
    vector<int> positions;
    if (m == 0 || m > n) return positions;

    long long patternHash = 0;
    long long windowHash = 0;
    long long highestPower = 1;

    for (int i = 0; i < m; i++) {
        patternHash = (patternHash * BASE + pattern[i]) % MOD;
        windowHash = (windowHash * BASE + text[i]) % MOD;
        if (i + 1 < m) highestPower = (highestPower * BASE) % MOD;
    }

    for (int start = 0; start + m <= n; start++) {
        if (patternHash == windowHash && text.compare(start, m, pattern) == 0) {
            positions.push_back(start);
        }

        if (start + m < n) {
            windowHash = (windowHash - text[start] * highestPower) % MOD;
            windowHash = (windowHash + text[start + m]) % MOD;
            highestPower = (highestPower * BASE) % MOD;
        }
    }

    return positions;
}
```

```

        windowHash = (windowHash - text[start] * highestPower % MOD + MOD) %
MOD;
        windowHash = (windowHash * BASE + text[start + m]) % MOD;
    }
}

return positions;
}

int main() {
    vector<int> positions = rabinKarpSearch("ababc", "ab");
    for (int pos : positions) cout << pos << " ";
    cout << "\n";
    return 0;
}

```

pYTHON IMPLEMENTATION

```

def rabin_karp_search(text: str, pattern: str) -> list[int]:
    mod = 1_000_000_007
    base = 911_382_323
    n, m = len(text), len(pattern)
    if m == 0 or m > n:
        return []

    pattern_hash = 0
    window_hash = 0
    highest_power = 1

    for i in range(m):
        pattern_hash = (pattern_hash * base + ord(pattern[i])) % mod
        window_hash = (window_hash * base + ord(text[i])) % mod
        if i + 1 < m:
            highest_power = (highest_power * base) % mod

    answer = []
    for start in range(n - m + 1):
        if pattern_hash == window_hash and text[start:start + m] == pattern:
            answer.append(start)

        if start + m < n:
            window_hash = (window_hash - ord(text[start]) * highest_power) % mod
            window_hash = (window_hash * base + ord(text[start + m])) % mod

    return answer

```

10. Code Explanation

- The hash stores characters as polynomial digits.
- `highestPower` is used to remove the outgoing leftmost character.

- After removal, multiply by `BASE` and add the incoming character.
- The code verifies with `text.compare` to avoid false positives from collisions.

11. Complexity Analysis

Case	Time	Space
Average with rare verification	$O(n + m)$	$O(1)$
Worst case with many collisions	$O(nm)$	$O(1)$
Double hash version	$O(n + m)$ expected	$O(1)$

12. Common Patterns

Pattern	Identify It	Approach	Examples
Exact matching	Pattern in text	Rolling hash windows	Implement strStr
Duplicate substring	Repeated substring length	Binary search + hash set	Longest Duplicate Substring
Multiple same-length patterns	Many fixed patterns	Hash all patterns	Plagiarism-style tasks

13. Common Mistakes

- Not adding `MOD` after subtraction.
- Forgetting collision verification.
- Choosing weak base/mod.
- Overflow in multiplication when using `int`.
- Wrong highest power.

14. Edge Cases

- Empty pattern.
- Pattern longer than text.
- All same characters.
- Hash collision.
- Large input.
- Non-lowercase characters.

15. Variations

Variation	Use
Single hash	Simple implementation
Double hash	Lower collision probability
64-bit unsigned hash	Fast CP technique

Variation	Use
Binary search + hash	Longest duplicate substring

16. Related Algorithms/Data Structures

- Rolling hash: general reusable hashing technique.
- KMP/Z: deterministic exact matching.
- Suffix array: deterministic duplicate substring solution.
- Hash set: stores window hashes.

17. Practice Problems

Easy

Problem	Platform	Main Idea	Difficulty
Implement strStr	LeetCode	Pattern search	Easy
Repeated Substring Pattern	LeetCode	Hash or KMP	Easy

Medium

Problem	Platform	Main Idea	Difficulty
Repeated DNA Sequences	LeetCode	Rolling fixed window	Medium
Find All Anagrams in a String	LeetCode	Window comparison	Medium

Hard

Problem	Platform	Main Idea	Difficulty
Longest Duplicate Substring	LeetCode	Binary search + hash	Hard
String Matching with Many Queries	Codeforces	Precomputed hashes	Hard

18. Interview Explanation

"Rabin-Karp compares the pattern hash with rolling hashes of text windows. Each slide removes the left character and adds the next one in constant time. Since hashes can collide, I either verify matching windows or use double hashing."

19. Revision Notes

- Rolling formula: remove old high-power char, shift, add new char.
- Use `long long`.
- Add modulo after subtraction.
- Verify on hash match unless double hashing is accepted.
- Average $O(n + m)$.

20. Final Cheat Sheet

Item	Note
Use when	Fixed-length substring matching/comparison
Main operation	Rolling hash update
Complexity	Average $O(n + m)$
Key code idea	$hash = (hash - old * p) * base + new$
Edge cases	Collision, empty pattern, modulo subtraction

Rolling Hash

1. Overview

Rolling hash is a technique for computing hashes of substrings quickly after preprocessing prefix hashes.

2. Intuition

Represent the string as a polynomial. Prefix hashes let you subtract the contribution before a substring and normalize the result.

3. When to Use It

Use rolling hash for:

- fast substring equality queries
- duplicate substring detection
- palindrome checking with forward/reverse hashes
- binary search on answer length
- Rabin-Karp style matching

4. When Not to Use It

Avoid it when:

- Exact deterministic guarantees are required and collision handling is not allowed.
- Only one direct comparison is needed.
- You need prefix dictionary behavior; use trie.
- You need all suffixes ordered; use suffix array.

5. Core Concepts

Concept	Meaning	Why It Matters
Prefix hash	Hash of $s[0..i-1]$	Allows substring hash
Power array	$base^i$ values	Removes/aligns substrings

Concept	Meaning	Why It Matters
Normalization	Make hash independent of position	Compare substrings
Double hash	Two mod values	Reduces collision risk

6. Step-by-Step Algorithm

- 1. Choose **BASE** and **MOD**.
- 2. Precompute powers of base.
- 3. Build prefix hashes where $pref[i+1] = pref[i] * BASE + s[i]$.
- 4. For substring $[l, r]$, compute $pref[r+1] - pref[l] * power[r-l+1]$.
- 5. Compare hashes for substring equality.

7. Dry Run

String: **abcd**

Prefix	Meaning
pref[0]	empty
pref[1]	hash(a)
pref[2]	hash(ab)
pref[3]	hash(abc)
pref[4]	hash(abcd)

Substring **bc** = $pref[3] - pref[1] * base^2$.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

class RollingHash {
private:
    static const long long MOD = 1000000007LL;
    static const long long BASE = 911382323LL;
    vector<long long> pref;
    vector<long long> power;

public:
    RollingHash(const string& s) {
        int n = s.size();
        pref.assign(n + 1, 0);
        power.assign(n + 1, 1);

        for (int i = 0; i < n; i++) {
            power[i + 1] = (power[i] * BASE) % MOD;
```

```

        pref[i + 1] = (pref[i] * BASE + s[i]) % MOD;
    }
}

long long getHash(int left, int right) const {
    long long value = (pref[right + 1] - pref[left] * power[right - left + 1])
% MOD;
    if (value < 0) value += MOD;
    return value;
}
};

int main() {
    string s = "ababc";
    RollingHash hash(s);
    cout << boolalpha << (hash.getHash(0, 1) == hash.getHash(2, 3)) << "\n";
    return 0;
}

```

pYTHON IMPLEMENTATION

```

class RollingHash:
    MOD = 1_000_000_007
    BASE = 911_382_323

    def __init__(self, s: str):
        self.pref = [0] * (len(s) + 1)
        self.power = [1] * (len(s) + 1)

        for i, ch in enumerate(s):
            self.power[i + 1] = (self.power[i] * self.BASE) % self.MOD
            self.pref[i + 1] = (self.pref[i] * self.BASE + ord(ch)) % self.MOD

    def get_hash(self, left: int, right: int) -> int:
        value = self.pref[right + 1] - self.pref[left] * self.power[right - left +
1]
        return value % self.MOD

```

10. Code Explanation

- `pref[i]` is the hash of the first `i` characters.
- `power[len]` is used to shift the left prefix before subtraction.
- `getHash(left, right)` returns the hash of that substring.
- Negative modulo is corrected in C++.

11. Complexity Analysis

Operation	Time	Space
Preprocessing	$O(n)$	$O(n)$
Substring hash query	$O(1)$	$O(1)$
Equality query	$O(1)$ expected	$O(1)$

12. Common Patterns

Pattern	Identify It	Approach	Examples
Substring equality	Many compare queries	Prefix hashes	Codeforces string queries
Palindrome query	Compare forward and reverse hash	Two hashes	Palindrome Queries
Longest duplicate	Check repeated length	Binary search + set	Longest Duplicate Substring

13. Common Mistakes

- Wrong substring formula.
- Forgetting modulo correction.
- Using one hash in adversarial problems.
- Not converting characters consistently.
- Comparing hashes of different lengths.

14. Edge Cases

- Empty string.
- Single-character substring.
- Full string substring.
- Same hash collision.
- Very long input.
- Invalid query bounds.

15. Variations

Variation	Use
Single modulo	Basic
Double modulo	Safer
<code>uint64_t</code> overflow hash	Fast CP
Forward + reverse hash	Palindrome queries

16. Related Algorithms/Data Structures

- Rabin-Karp: uses rolling hash for pattern matching.
- Suffix array: deterministic alternative for suffix/substrings.
- Manacher: better for palindrome radii.
- Hash set: stores substring hashes.

17. Practice Problems

Easy

Problem	Platform	Main Idea	Difficulty
Implement strStr	LeetCode	Hash matching	Easy
Repeated Substring Pattern	LeetCode	Hash repetition	Easy

Medium

Problem	Platform	Main Idea	Difficulty
Repeated DNA Sequences	LeetCode	Fixed-length hashes	Medium
Palindrome Queries	GFG/Codeforces	Forward/reverse hash	Medium

Hard

Problem	Platform	Main Idea	Difficulty
Longest Duplicate Substring	LeetCode	Binary search + rolling hash	Hard
CSES Finding Periods	CSES	Hash or Z/KMP	Hard

18. Interview Explanation

"Rolling hash preprocesses prefix hashes and powers so I can get any substring hash in constant time. This is useful for many substring comparisons, but because hashes can collide, I use double hashing or verification when correctness must be guaranteed."

19. Revision Notes

- Formula: $\text{hash}(l, r) = \text{pref}[r+1] - \text{pref}[l] * \text{pow}[len]$.
- Fix negative modulo.
- Precompute powers.
- Double hash for safety.
- Query is $O(1)$ after $O(n)$.

20. Final Cheat Sheet

Item	Note
Use when	Many substring comparisons

Item	Note
Main operation	Prefix hash query
Complexity	$O(n)$ build, $O(1)$ query
Key code idea	Subtract shifted prefix
Edge cases	Collision, bounds, negative mod

Trie

1. Overview

A trie is a tree data structure for storing strings character by character. It is especially useful for prefix queries.

2. Intuition

Words with the same prefix share the same path. For example, **cat**, **car**, and **care** share **ca**, so the trie stores that prefix once.

3. When to Use It

Use a trie when:

- "prefix search"
- "autocomplete"
- "dictionary words"
- "starts with"
- "word break"
- "maximum XOR" with binary representation
- "replace words by root"

4. When Not to Use It

Avoid it when:

- You only need exact lookup once; a hash set is simpler.
- Memory is tight and alphabet is large.
- You need substring search; suffix automaton/array may be better.
- You need many patterns in text; use Aho-Corasick.

5. Core Concepts

Concept	Meaning	Why It Matters
Node	Represents a prefix	Stores children
Edge	Character transition	Builds words
isEnd	Marks complete word	Distinguishes word from prefix

Concept	Meaning	Why It Matters
Prefix sharing	Common paths reused	Saves repeated prefixes

6. Step-by-Step Algorithm

Insert:

1. Start at root.
2. For each character, move to its child.
3. Create child if missing.
4. Mark final node as word end.

Search:

1. Start at root.
2. Follow each character.
3. If a transition is missing, return false.
4. For full word search, check `isEnd`.

7. Dry Run

Insert: `cat`, `car`

Word	Path Created/Used
cat	c -> a -> t
car	c and a reused, r created

`ca` is shared by both words.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

class Trie {
private:
    struct Node {
        array<int, 26> next;
        bool isEnd;

        Node() {
            next.fill(-1);
            isEnd = false;
        }
    };

    vector<Node> nodes;
```

```

public:
    Trie() {
        nodes.push_back(Node());
    }

    void insert(const string& word) {
        int current = 0;
        for (char ch : word) {
            int c = ch - 'a';
            if (nodes[current].next[c] == -1) {
                nodes[current].next[c] = nodes.size();
                nodes.push_back(Node());
            }
            current = nodes[current].next[c];
        }
        nodes[current].isEnd = true;
    }

    bool search(const string& word) const {
        int node = walk(word);
        return node != -1 && nodes[node].isEnd;
    }

    bool startsWith(const string& prefix) const {
        return walk(prefix) != -1;
    }

private:
    int walk(const string& s) const {
        int current = 0;
        for (char ch : s) {
            int c = ch - 'a';
            if (nodes[current].next[c] == -1) return -1;
            current = nodes[current].next[c];
        }
        return current;
    }
};

int main() {
    Trie trie;
    trie.insert("apple");
    cout << boolalpha << trie.search("apple") << "\n";
    cout << trie.search("app") << "\n";
    cout << trie.startsWith("app") << "\n";
    return 0;
}

```

pYTHON IMPLEMENTATION

```
class TrieNode:
    def __init__(self):
        self.children = {}
        self.is_end = False

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word: str) -> None:
        node = self.root
        for ch in word:
            node = node.children.setdefault(ch, TrieNode())
        node.is_end = True

    def search(self, word: str) -> bool:
        node = self._walk(word)
        return node is not None and node.is_end

    def starts_with(self, prefix: str) -> bool:
        return self._walk(prefix) is not None

    def _walk(self, s: str):
        node = self.root
        for ch in s:
            if ch not in node.children:
                return None
            node = node.children[ch]
        return node
```

10. Code Explanation

- Each node stores child transitions and whether a word ends there.
- C++ uses `array<int, 26>` for compact lowercase transitions.
- `walk` avoids duplicate traversal logic for `search` and `startsWith`.
- Python uses dictionaries, better for flexible alphabets.

11. Complexity Analysis

Operation	Time	Space
Insert word length L	$O(L)$	New nodes up to $O(L)$
Search word	$O(L)$	$O(1)$ extra
Prefix query	$O(L)$	$O(1)$ extra
Build all words	$O(\text{total characters})$	$O(\text{total characters} * \text{alphabet storage})$

12. Common Patterns

Pattern	Identify It	Approach	Examples
Prefix lookup	starts with	Trie walk	Implement Trie
Autocomplete	suggestions by prefix	Trie + DFS/list	Search Suggestions
Word board search	dictionary + grid	Trie + DFS	Word Search II

13. Common Mistakes

- Forgetting `isEnd`, making prefix equal to word.
- Assuming lowercase input.
- Huge memory usage with large fixed arrays.
- Not handling duplicate insertions if counts matter.
- Recursion depth issues in DFS over large tries.

14. Edge Cases

- Empty string insertion.
- Search prefix that is not a word.
- Duplicate words.
- Uppercase or symbols.
- Very large dictionary.

15. Variations

Variation	Use
Fixed-array trie	Small alphabet, fast
Hash-map trie	Large/sparse alphabet
Compressed trie	Memory optimization
Binary trie	XOR problems
Aho-Corasick trie	Multi-pattern matching

16. Related Algorithms/Data Structures

- Hash set: exact lookup, simpler.
- Aho-Corasick: trie plus failure links for text scanning.
- Suffix tree/automaton: substring-oriented structures.
- Sorting + binary search: prefix queries with sorted words.

17. Practice Problems

Easy

Problem	Platform	Main Idea	Difficulty
Implement Trie	LeetCode	Insert/search/prefix	Medium but basic

Problem	Platform	Main Idea	Difficulty
Longest Common Prefix	LeetCode	Trie alternative	Easy

Medium

Problem	Platform	Main Idea	Difficulty
Design Add and Search Words	LeetCode	Trie + wildcard DFS	Medium
Replace Words	LeetCode	Root prefix search	Medium
Search Suggestions System	LeetCode	Prefix suggestions	Medium

Hard

Problem	Platform	Main Idea	Difficulty
Word Search II	LeetCode	Trie + backtracking	Hard
Maximum XOR With an Element From Array	LeetCode	Binary trie	Hard

18. Interview Explanation

"A trie stores words by sharing common prefixes. Each node represents a prefix, and edges represent characters. Insert, search, and prefix queries take time proportional to the word length, which makes tries strong for dictionary and autocomplete problems."

19. Revision Notes

- Node = prefix.
- `isEnd` distinguishes full word from prefix.
- Insert/search are $O(L)$.
- Array trie for lowercase, map trie for sparse alphabet.
- Memory can be high.

20. Final Cheat Sheet

Item	Note
Use when	Prefix/dictionary queries
Main operation	Walk character edges
Complexity	$O(L)$ per operation
Key code idea	Create missing child while inserting
Edge cases	Prefix not word, duplicates, alphabet

Manacher's Algorithm

1. Overview

Manacher's algorithm finds palindromic radii around every center in linear time. It is mainly used for longest palindromic substring and counting palindromic substrings efficiently.

2. Intuition

Palindromes mirror around their center. If you know a large palindrome $[l, r]$, then a center inside it has a mirrored center whose palindrome radius gives a starting estimate.

This avoids expanding from every center from scratch.

3. When to Use It

Use Manacher when:

- "longest palindromic substring" with large constraints
- "count palindromic substrings" in $O(n)$
- many palindrome radius queries
- CP constraints require linear time

4. When Not to Use It

Avoid it when:

- $O(n^2)$ center expansion is accepted and simpler.
- You need palindromic subsequence, not substring.
- You need dynamic online insertion; use Eertree.
- Implementation risk is too high for a basic interview.

5. Core Concepts

Concept	Meaning	Why It Matters
Radius	How far palindrome expands from center	Gives substring length
Odd palindrome	One-character center	aba
Even palindrome	Gap center	abba
Mirror center	Symmetric center inside known palindrome	Reuses work

6. Step-by-Step Algorithm

Odd radii:

1. Maintain current palindrome window $[left, right]$.
2. For each center i , set initial radius using mirror if inside window.
3. Expand while characters match.
4. Store radius.
5. Update $[left, right]$ if palindrome extends farther.

Repeat similarly for even radii.

7. Dry Run

String: **ababa**

Odd radii:

Center	Char	Radius	Palindrome
0	a	1	a
1	b	2	aba
2	a	3	ababa
3	b	2	aba
4	a	1	a

Longest palindrome: **ababa**.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

vector<int> manacherOdd(const string& s) {
    int n = s.size();
    vector<int> d1(n);
    int left = 0, right = -1;

    for (int i = 0; i < n; i++) {
        int radius = (i > right) ? 1 : min(d1[left + right - i], right - i + 1);

        while (i - radius >= 0 && i + radius < n && s[i - radius] == s[i + radius]) {
            radius++;
        }

        d1[i] = radius;

        if (i + radius - 1 > right) {
            left = i - radius + 1;
            right = i + radius - 1;
        }
    }

    return d1;
}

vector<int> manacherEven(const string& s) {
    int n = s.size();
```

```

vector<int> d2(n);
int left = 0, right = -1;

for (int i = 0; i < n; i++) {
    int radius = (i > right) ? 0 : min(d2[left + right - i + 1], right - i + 1);

    while (i - radius - 1 >= 0 && i + radius < n &&
           s[i - radius - 1] == s[i + radius]) {
        radius++;
    }

    d2[i] = radius;

    if (i + radius - 1 > right) {
        left = i - radius;
        right = i + radius - 1;
    }
}

return d2;
}

string longestPalindromicSubstring(const string& s) {
    if (s.empty()) return "";

    vector<int> d1 = manacherOdd(s);
    vector<int> d2 = manacherEven(s);
    int bestLength = 1, bestStart = 0;

    for (int i = 0; i < (int)s.size(); i++) {
        int oddLength = 2 * d1[i] - 1;
        if (oddLength > bestLength) {
            bestLength = oddLength;
            bestStart = i - d1[i] + 1;
        }

        int evenLength = 2 * d2[i];
        if (evenLength > bestLength) {
            bestLength = evenLength;
            bestStart = i - d2[i];
        }
    }

    return s.substr(bestStart, bestLength);
}

int main() {
    cout << longestPalindromicSubstring("babad") << "\n";
    return 0;
}

```

pYTHON IMPLEMENTATION

```
def manacher_odd(s: str) -> list[int]:
    n = len(s)
    d1 = [0] * n
    left, right = 0, -1

    for i in range(n):
        radius = 1 if i > right else min(d1[left + right - i], right - i + 1)
        while i - radius >= 0 and i + radius < n and s[i - radius] == s[i + radius]:
            radius += 1
        d1[i] = radius
        if i + radius - 1 > right:
            left = i - radius + 1
            right = i + radius - 1

    return d1

def manacher_even(s: str) -> list[int]:
    n = len(s)
    d2 = [0] * n
    left, right = 0, -1

    for i in range(n):
        radius = 0 if i > right else min(d2[left + right - i + 1], right - i + 1)
        while (
            i - radius - 1 >= 0
            and i + radius < n
            and s[i - radius - 1] == s[i + radius]
        ):
            radius += 1
        d2[i] = radius
        if i + radius - 1 > right:
            left = i - radius
            right = i + radius - 1

    return d2

def longest_palindromic_substring(s: str) -> str:
    if not s:
        return ""

    d1 = manacher_odd(s)
    d2 = manacher_even(s)
    best_start, best_len = 0, 1

    for i in range(len(s)):
        odd_len = 2 * d1[i] - 1
        if odd_len > best_len:
```

```
        best_len = odd_len
        best_start = i - d1[i] + 1

    even_len = 2 * d2[i]
    if even_len > best_len:
        best_len = even_len
        best_start = i - d2[i]

    return s[best_start:best_start + best_len]
```

10. Code Explanation

- `d1[i]` stores odd palindrome radius centered at `i`, including the center.
- `d2[i]` stores even palindrome radius centered between `i-1` and `i`.
- Mirror indices initialize a safe lower bound for radius.
- Expansion only happens beyond the known right boundary, giving linear time.

11. Complexity Analysis

Operation	Time	Space
Odd radii	$O(n)$	$O(n)$
Even radii	$O(n)$	$O(n)$
Longest palindrome	$O(n)$	$O(n)$

12. Common Patterns

Pattern	Identify It	Approach	Examples
Longest palindrome substring	Large <code>n</code>	Manacher radii	LeetCode Longest Palindromic Substring
Count pal substrings	Need $O(n)$	Sum radii	Palindromic Substrings
Pal radius query	Need radius per center	Use <code>d1/d2</code>	CP tasks

13. Common Mistakes

- Mixing odd and even radius definitions.
- Wrong mirror index.
- Off-by-one in start/length conversion.
- Using Manacher for subsequences.
- Forgetting empty string.

14. Edge Cases

- Empty string.
- Single character.

- All same characters.
- Even palindrome like **abba**.
- Odd palindrome like **aba**.
- No palindrome longer than 1.

15. Variations

Variation	Use
Separate odd/even arrays	Clear CP style
Transformed string with separators	Single radius array
Count palindromes	Sum radii
Palindrome query helper	Use radii to test centers

16. Related Algorithms/Data Structures

- Center expansion: simpler $O(n^2)$.
- Eertree: stores distinct palindromes online.
- Rolling hash: palindrome checks with collision risk.
- DP: palindromic substring table.

17. Practice Problems

Easy

Problem	Platform	Main Idea	Difficulty
Valid Palindrome	LeetCode	Basic palindrome	Easy
Palindrome Number	LeetCode	Symmetry	Easy

Medium

Problem	Platform	Main Idea	Difficulty
Longest Palindromic Substring	LeetCode	Manacher or center expand	Medium
Palindromic Substrings	LeetCode	Sum centers/radii	Medium

Hard

Problem	Platform	Main Idea	Difficulty
Palindrome Queries	Codeforces	Radii/hash	Hard
Eertree-based palindrome tasks	Codeforces	Palindromic structure	Hard

18. Interview Explanation

"Manacher computes the palindrome radius around every center in linear time. It reuses the symmetry of the current rightmost palindrome, so centers inside it get an initial radius from their mirror. Only expansions beyond the known boundary cost new comparisons."

19. Revision Notes

- **d1**: odd radius including center.
- **d2**: even radius around gap.
- Longest odd length: $2*d1[i]-1$.
- Longest even length: $2*d2[i]$.
- Trap: off-by-one boundaries.

20. Final Cheat Sheet

Item	Note
Use when	Longest/count palindromic substrings in linear time
Main operation	Mirror radius inside rightmost palindrome
Complexity	$O(n)$
Key code idea	Maintain <code>[left, right]</code>
Edge cases	Empty, even palindromes, all same chars

Suffix Array

1. Overview

A suffix array is a sorted array of all suffix starting positions of a string. It supports many substring and lexicographic queries.

2. Intuition

All substrings are prefixes of suffixes. If suffixes are sorted, repeated substrings and lexicographic relationships become easier to find.

3. When to Use It

Use suffix array when:

- count distinct substrings
- longest repeated substring
- lexicographic substring/suffix queries
- many pattern existence queries
- need deterministic alternative to hashing

4. When Not to Use It

Avoid it when:

- A single pattern search can be solved with KMP/Z.
- Prefix dictionary queries need trie.
- Online updates are required.
- Implementation time is limited and constraints allow simpler methods.

5. Core Concepts

Concept	Meaning	Why It Matters
Suffix	Substring from index i to end	Sorting unit
Suffix array sa	Starting indices in sorted suffix order	Core structure
LCP array	LCP of adjacent sorted suffixes	Repetition/distinct counts
Doubling	Sort by length 2^k prefixes	Efficient construction

6. Step-by-Step Algorithm

Doubling construction:

1. Append sentinel **\$** smaller than all characters.
2. Sort suffixes by first character.
3. Assign equivalence classes.
4. Repeatedly sort by pairs of classes representing length **2^k**.
5. Double **k** until length covers the string.
6. Remove sentinel suffix if needed.

Kasai LCP:

1. Build rank array from suffix array.
2. For each index, compare with previous suffix in sorted order.
3. Reuse previous LCP minus one.

7. Dry Run

String: **banana**

Sorted suffixes:

Rank	Index	Suffix
0	5	a
1	3	ana
2	1	anana
3	0	banana
4	4	na

Rank	Index	Suffix
5	2	nana

Suffix array: [5,3,1,0,4,2].

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

vector<int> buildSuffixArray(string s) {
    s += '$';
    int n = s.size();
    vector<int> sa(n), classes(n);

    vector<pair<char, int>> initial(n);
    for (int i = 0; i < n; i++) initial[i] = {s[i], i};
    sort(initial.begin(), initial.end());

    for (int i = 0; i < n; i++) sa[i] = initial[i].second;
    classes[sa[0]] = 0;
    for (int i = 1; i < n; i++) {
        classes[sa[i]] = classes[sa[i - 1]] + (initial[i].first != initial[i - 1].first);
    }

    for (int k = 0; (1 << k) < n; k++) {
        for (int i = 0; i < n; i++) {
            sa[i] = (sa[i] - (1 << k) + n) % n;
        }

        vector<int> count(n), newSa(n), newClasses(n);
        for (int c : classes) count[c]++;
        for (int i = 1; i < n; i++) count[i] += count[i - 1];
        for (int i = n - 1; i >= 0; i--) {
            newSa[--count[classes[sa[i]]]] = sa[i];
        }
        sa = newSa;

        newClasses[sa[0]] = 0;
        for (int i = 1; i < n; i++) {
            pair<int, int> prev = {classes[sa[i - 1]], classes[(sa[i - 1] + (1 << k)) % n]};
            pair<int, int> cur = {classes[sa[i]], classes[(sa[i] + (1 << k)) % n]};
            newClasses[sa[i]] = newClasses[sa[i - 1]] + (cur != prev);
        }
        classes = newClasses;
    }

    sa.erase(sa.begin());
}
```

```

        return sa;
    }

    vector<int> buildLCP(const string& s, const vector<int>& sa) {
        int n = s.size();
        vector<int> rank(n), lcp(n - 1);
        for (int i = 0; i < n; i++) rank[sa[i]] = i;

        int k = 0;
        for (int i = 0; i < n; i++) {
            if (rank[i] == n - 1) {
                k = 0;
                continue;
            }
            int j = sa[rank[i] + 1];
            while (i + k < n && j + k < n && s[i + k] == s[j + k]) k++;
            lcp[rank[i]] = k;
            if (k > 0) k--;
        }
        return lcp;
    }

    int main() {
        string s = "banana";
        vector<int> sa = buildSuffixArray(s);
        vector<int> lcp = buildLCP(s, sa);
        for (int x : sa) cout << x << " ";
        cout << "\n";
        for (int x : lcp) cout << x << " ";
        cout << "\n";
        return 0;
    }
}

```

pYTHON IMPLEMENTATION

```

def build_suffix_array(s: str) -> list[int]:
    return sorted(range(len(s)), key=lambda i: s[i:])

def build_lcp(s: str, sa: list[int]) -> list[int]:
    n = len(s)
    rank = [0] * n
    for i, suffix_start in enumerate(sa):
        rank[suffix_start] = i

    lcp = [0] * max(0, n - 1)
    k = 0
    for i in range(n):
        if rank[i] == n - 1:
            k = 0
            continue

```

```

    j = sa[rank[i] + 1]
    while i + k < n and j + k < n and s[i + k] == s[j + k]:
        k += 1
    lcp[rank[i]] = k
    if k:
        k -= 1
    return lcp

```

10. Code Explanation

- The C++ suffix array uses the doubling method and counting sort by equivalence classes.
- `classes[i]` identifies the rank class of the cyclic substring starting at `i`.
- Shifting suffixes left by 2^k lets counting sort by the first half after the second half is already ordered.
- Kasai's algorithm builds LCP in linear time by reusing the previous match length.
- The Python suffix array is simple but $O(n^2 \log n)$ due to substring comparisons; use only for learning or small inputs.

11. Complexity Analysis

Operation	Time	Space
C++ suffix array	$O(n \log n)$	$O(n)$
Kasai LCP	$O(n)$	$O(n)$
Pattern binary search	$O(m \log n)$ with careful compare	$O(1)$
Count distinct substrings	$O(n)$ after SA/LCP	$O(n)$

12. Common Patterns

Pattern	Identify It	Approach	Examples
Distinct substrings	Count unique substrings	$\frac{n(n+1)}{2} - \text{sum}(\text{lcp})$	CSES Distinct Substrings
Longest repeated substring	Max repeated block	$\text{max}(\text{lcp})$	LeetCode Longest Duplicate
Pattern exists	Query pattern in text	Binary search suffix array	CSES Pattern Positions

13. Common Mistakes

- Forgetting sentinel.
- Sentinel not lexicographically smallest.
- Confusing cyclic shifts with suffixes after removing sentinel.
- Off-by-one in LCP size.
- Python naive suffix array TLE on large inputs.

14. Edge Cases

- Empty string.
- Single character.
- All same characters.
- All unique characters.
- Very long repeated string.
- Sentinel character appears in input.

15. Variations

Variation	Use
Doubling suffix array	Standard CP
SA-IS	Linear construction, advanced
Suffix array + LCP + RMQ	LCP queries
Generalized suffix array	Multiple strings

16. Related Algorithms/Data Structures

- Suffix automaton: often simpler for distinct substring count and online extension.
- Suffix tree: powerful but complex.
- Rolling hash: simpler but probabilistic.
- Z/KMP: easier for single pattern search.

17. Practice Problems

Easy

Problem	Platform	Main Idea	Difficulty
Suffix Array Learning Task	GFG	Build sorted suffixes	Easy
Longest Common Prefix	LeetCode	LCP concept	Easy

Medium

Problem	Platform	Main Idea	Difficulty
CSES Distinct Substrings	CSES	SA + LCP	Medium
CSES Finding Patterns	CSES	Binary search SA	Medium

Hard

Problem	Platform	Main Idea	Difficulty
Longest Duplicate Substring	LeetCode	SA/LCP or hash	Hard
Codeforces suffix array tasks	Codeforces	SA + LCP/RMQ	Hard

18. Interview Explanation

"A suffix array stores all suffix starting indices in lexicographic order. Since every substring is a prefix of some suffix, sorted suffixes make repeated and distinct substring problems easier. The LCP array gives common prefixes of adjacent suffixes, which is key for counting duplicates."

19. Revision Notes

- `sa` = sorted suffix start indices.
- `lcp[i]` = LCP of `sa[i]` and `sa[i+1]`.
- Distinct substrings: $n(n+1)/2 - \text{sum}(\text{lcp})$.
- Build in $O(n \log n)$.
- Sentinel must be smallest.

20. Final Cheat Sheet

Item	Note
Use when	Sorted suffix/substring queries
Main operation	Sort suffixes, compute LCP
Complexity	$O(n \log n)$ build
Key code idea	Doubling classes
Edge cases	Sentinel, repeated chars, empty string

Suffix Automaton

1. Overview

A suffix automaton (SAM) is a compact state machine representing all substrings of a string. It is powerful for counting distinct substrings and solving substring occurrence problems.

2. Intuition

As you append characters, SAM groups substrings that have the same possible future continuations. Each state represents a set of substrings with related ending positions.

Less formal: SAM is like a compressed map of every substring in the string.

3. When to Use It

Use suffix automaton when:

- count distinct substrings
- longest common substring of two strings
- number of occurrences of each substring
- online construction is useful
- advanced CP string problems

4. When Not to Use It

Avoid it when:

- You only need exact pattern matching; use KMP/Z.
- You only need prefix queries; use trie.
- The problem is interview-level and simpler suffix array/hash works.
- You are not comfortable with clones; implementation bugs are common.

5. Core Concepts

Concept	Meaning	Why It Matters
State	Represents many substrings	Compact storage
len	Longest string in state	Used for counts
Link	Suffix link to smaller state	Failure-like transition
Transition	Add character	Automaton movement
Clone	Split state when needed	Maintains correctness

6. Step-by-Step Algorithm

For each character:

1. Create a new state `cur` with `len = last.len + 1`.
2. Walk suffix links from `last`, adding missing transitions by this character to `cur`.
3. If no previous state exists, link `cur` to root.
4. Otherwise inspect transition state `q`.
5. If `p.len + 1 == q.len`, link `cur` to `q`.
6. Else create clone of `q`, adjust links and transitions.
7. Set `last = cur`.

7. Dry Run

String: `ab`

Add	Effect
a	root --a--> state for a
b	add transitions for b; substrings represented: a, b, ab

Distinct substrings: 3.

Formula: sum over states except root of `len[state] - len[link[state]]`.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

class SuffixAutomaton {
private:
    struct State {
        int link = -1;
        int len = 0;
        map<char, int> next;
    };

    vector<State> st;
    int last;

public:
    SuffixAutomaton() {
        st.push_back(State());
        last = 0;
    }

    void extend(char c) {
        int cur = st.size();
        st.push_back(State());
        st[cur].len = st[last].len + 1;

        int p = last;
        while (p != -1 && !st[p].next.count(c)) {
            st[p].next[c] = cur;
            p = st[p].link;
        }

        if (p == -1) {
            st[cur].link = 0;
        } else {
            int q = st[p].next[c];
            if (st[p].len + 1 == st[q].len) {
                st[cur].link = q;
            } else {
                int clone = st.size();
                st.push_back(st[q]);
                st[clone].len = st[p].len + 1;

                while (p != -1 && st[p].next[c] == q) {
                    st[p].next[c] = clone;
                    p = st[p].link;
                }

                st[q].link = st[cur].link = clone;
            }
        }

        last = cur;
    }
};
```

```

    }

    void build(const string& s) {
        for (char c : s) extend(c);
    }

    long long countDistinctSubstrings() const {
        long long answer = 0;
        for (int v = 1; v < (int)st.size(); v++) {
            answer += st[v].len - st[st[v].link].len;
        }
        return answer;
    }

    bool contains(const string& pattern) const {
        int current = 0;
        for (char c : pattern) {
            if (!st[current].next.count(c)) return false;
            current = st[current].next.at(c);
        }
        return true;
    }
};

int main() {
    SuffixAutomaton sam;
    sam.build("ababa");
    cout << sam.countDistinctSubstrings() << "\n";
    cout << boolalpha << sam.contains("bab") << "\n";
    return 0;
}

```

pYTHON IMPLEMENTATION

```

class SuffixAutomaton:
    def __init__(self):
        self.link = [-1]
        self.length = [0]
        self.next = [dict()]
        self.last = 0

    def extend(self, c: str) -> None:
        cur = len(self.length)
        self.length.append(self.length[self.last] + 1)
        self.link.append(0)
        self.next.append({})

        p = self.last
        while p != -1 and c not in self.next[p]:
            self.next[p][c] = cur
            p = self.link[p]

```

```

    if p == -1:
        self.link[cur] = 0
    else:
        q = self.next[p][c]
        if self.length[p] + 1 == self.length[q]:
            self.link[cur] = q
        else:
            clone = len(self.length)
            self.length.append(self.length[p] + 1)
            self.link.append(self.link[q])
            self.next.append(self.next[q].copy())

            while p != -1 and self.next[p].get(c) == q:
                self.next[p][c] = clone
                p = self.link[p]

            self.link[q] = self.link[cur] = clone

    self.last = cur

def build(self, s: str) -> None:
    for c in s:
        self.extend(c)

def count_distinct_substrings(self) -> int:
    return sum(self.length[v] - self.length[self.link[v]] for v in range(1,
len(self.length)))

```

10. Code Explanation

- `last` is the state representing the whole current string.
- New state `cur` represents substrings ending after the new character.
- Suffix links are followed to add missing transitions.
- A clone is created when an existing transition state is too long and must be split.
- Distinct substring count comes from how many new lengths each state contributes.

11. Complexity Analysis

Operation	Time	Space
Build with map transitions	$O(n \log \text{alphabet})$	$O(n)$ states
Build with unordered_map	$O(n)$ average	$O(n)$
Contains query	$O(m)$	$O(1)$ extra
Count distinct substrings	$O(\text{number of states})$	$O(1)$ extra

12. Common Patterns

Pattern	Identify It	Approach	Examples
Count distinct substrings	Number of unique substrings	Sum $len - linklen$	CSES Distinct Substrings
Longest common substring	Compare two strings	Walk second string on SAM	GFG/CP tasks
Occurrence count	Frequency of substrings	DP over suffix links	Codeforces tasks

13. Common Mistakes

- Incorrect clone length.
- Forgetting to redirect transitions to clone.
- Using SAM for subsequences.
- Not initializing root link as -1 .
- Assuming states equal substrings one-to-one.

14. Edge Cases

- Empty string.
- Single character.
- All same characters.
- All unique characters.
- Large alphabet.
- Very long repeated patterns.

15. Variations

Variation	Use
SAM with occurrence counts	Count substring frequency
Generalized SAM	Multiple strings
SAM with DP	K-th substring, lexicographic tasks
Online SAM	Append characters dynamically

16. Related Algorithms/Data Structures

- Suffix array: deterministic sorted suffix approach.
- Suffix tree: tree version of suffix structure.
- Trie: prefix-only, simpler.
- Aho-Corasick: multiple pattern matching.

17. Practice Problems

Easy

Problem	Platform	Main Idea	Difficulty
Distinct Substring Count Small	GFG	Understand substrings	Easy
Implement Substring Search	LeetCode	Simpler alternative	Easy

Medium

Problem	Platform	Main Idea	Difficulty
CSES Distinct Substrings	CSES	SAM or suffix array	Medium
Longest Common Substring	GFG	SAM walk	Medium

Hard

Problem	Platform	Main Idea	Difficulty
Codeforces SAM tasks	Codeforces	Occurrence/DP on SAM	Hard
SPOJ SUBLEX	SPOJ	K-th substring via SAM	Hard

18. Interview Explanation

"A suffix automaton compactly represents all substrings of a string. Each state contributes a range of substring lengths, so counting distinct substrings is the sum of $\text{len}[\text{state}] - \text{len}[\text{link}[\text{state}]]$. It is advanced but very powerful for competitive programming substring problems."

19. Revision Notes

- Max states: about $2n - 1$.
- $\text{len}[v]$ = longest string in state.
- Contribution: $\text{len}[v] - \text{len}[\text{link}[v]]$.
- Clones split states.
- Not for subsequences.

20. Final Cheat Sheet

Item	Note
Use when	Advanced substring counting/queries
Main operation	Extend automaton by char
Complexity	$O(n)$ average
Key code idea	Create clone when transition length mismatch
Edge cases	Empty, all same, clone-heavy strings

Aho-Corasick

1. Overview

Aho-Corasick matches multiple patterns in a text in linear time after building a trie with failure links.

2. Intuition

It is like running KMP for many patterns at once. The trie follows matching characters; failure links tell where to continue after a mismatch without restarting.

3. When to Use It

Use Aho-Corasick when:

- multiple pattern matching
- dictionary words inside text
- banned words detection
- count occurrences of many patterns
- text is large and patterns are many

4. When Not to Use It

Avoid it when:

- There is only one pattern; KMP/Z is simpler.
- You need approximate matching.
- Patterns change frequently and rebuilding is expensive.
- You need prefix lookup only; trie is enough.

5. Core Concepts

Concept	Meaning	Why It Matters
Trie	Stores all patterns	Shared prefixes
Failure link	Best fallback state	Avoids restart
Output list	Patterns ending at state	Reports matches
Automaton transition	Next state for char	Fast scanning

6. Step-by-Step Algorithm

1. Insert all patterns into a trie.
2. BFS from root to build failure links.
3. For missing transitions, point to fallback transitions.
4. Scan text character by character through the automaton.
5. At each state, report all pattern IDs in its output list.

7. Dry Run

Patterns: **he**, **she**, **his**, **hers**; Text: **ushers**

Text Index	Char	Matches Ending Here
1	s	none
2	h	none
3	e	she, he
5	s	hers

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

class AhoCorasick {
private:
    struct Node {
        array<int, 26> next;
        int link;
        vector<int> output;

        Node() {
            next.fill(-1);
            link = 0;
        }
    };

    vector<Node> trie;

public:
    AhoCorasick() {
        trie.push_back(Node());
    }

    void addPattern(const string& pattern, int id) {
        int current = 0;
        for (char ch : pattern) {
            int c = ch - 'a';
            if (trie[current].next[c] == -1) {
                trie[current].next[c] = trie.size();
                trie.push_back(Node());
            }
            current = trie[current].next[c];
        }
        trie[current].output.push_back(id);
    }

    void build() {
        queue<int> q;

        for (int c = 0; c < 26; c++) {
            int child = trie[0].next[c];
```

```

        if (child == -1) {
            trie[0].next[c] = 0;
        } else {
            trie[child].link = 0;
            q.push(child);
        }
    }

    while (!q.empty()) {
        int v = q.front();
        q.pop();

        for (int id : trie[trie[v].link].output) {
            trie[v].output.push_back(id);
        }

        for (int c = 0; c < 26; c++) {
            int child = trie[v].next[c];
            if (child == -1) {
                trie[v].next[c] = trie[trie[v].link].next[c];
            } else {
                trie[child].link = trie[trie[v].link].next[c];
                q.push(child);
            }
        }
    }
}

vector<pair<int, int>> search(const string& text, const vector<string>&
patterns) const {
    vector<pair<int, int>> matches;
    int current = 0;

    for (int i = 0; i < (int)text.size(); i++) {
        int c = text[i] - 'a';
        current = trie[current].next[c];

        for (int id : trie[current].output) {
            matches.push_back({i - (int)patterns[id].size() + 1, id});
        }
    }

    return matches;
}

};

int main() {
    vector<string> patterns = {"he", "she", "his", "hers"};
    AhoCorasick ac;
    for (int i = 0; i < (int)patterns.size(); i++) ac.addPattern(patterns[i], i);
    ac.build();

    auto matches = ac.search("ushers", patterns);
    for (auto [pos, id] : matches) {

```

```

        cout << patterns[id] << " at " << pos << "\n";
    }
    return 0;
}

```

pYTHON IMPLEMENTATION

```

from collections import deque

class AhoCorasick:
    def __init__(self):
        self.next = [[-1] * 26]
        self.link = [0]
        self.output = [[]]

    def add_pattern(self, pattern: str, pattern_id: int) -> None:
        node = 0
        for ch in pattern:
            c = ord(ch) - ord("a")
            if self.next[node][c] == -1:
                self.next[node][c] = len(self.next)
                self.next.append([-1] * 26)
                self.link.append(0)
                self.output.append([])
            node = self.next[node][c]
        self.output[node].append(pattern_id)

    def build(self) -> None:
        q = deque()
        for c in range(26):
            child = self.next[0][c]
            if child == -1:
                self.next[0][c] = 0
            else:
                q.append(child)

        while q:
            v = q.popleft()
            self.output[v].extend(self.output[self.link[v]])
            for c in range(26):
                child = self.next[v][c]
                if child == -1:
                    self.next[v][c] = self.next[self.link[v]][c]
                else:
                    self.link[child] = self.next[self.link[v]][c]
                    q.append(child)

```

10. Code Explanation

- Patterns are inserted into a trie.
- `link` points to the longest proper suffix state that is also a trie prefix.
- BFS builds failure links level by level.
- Missing transitions are filled, so scanning never needs a manual fallback loop.
- Output from failure states is inherited so suffix pattern matches are reported.

11. Complexity Analysis

Operation	Time	Space
Build trie	$O(\text{total pattern length})$	$O(\text{nodes} * \text{alphabet})$
Build failure links	$O(\text{nodes} * \text{alphabet})$	$O(\text{nodes} * \text{alphabet})$
Scan text	$O(\text{text length} + \text{matches})$	$O(1)$ extra

12. Common Patterns

Pattern	Identify It	Approach	Examples
Many patterns in text	Dictionary matching	Aho-Corasick	Multi-pattern search
Banned words	Detect any word	Stop on first output	Spam filters
Count pattern occurrences	Need all counts	Accumulate output matches	CP dictionary tasks

13. Common Mistakes

- Not propagating output through failure links.
- Assuming lowercase input.
- Forgetting to build before searching.
- Memory explosion with large alphabet arrays.
- Incorrect match start index.

14. Edge Cases

- Empty pattern list.
- Pattern longer than text.
- One pattern is suffix of another.
- Duplicate patterns.
- Text contains unsupported characters.
- Many matches at one position.

15. Variations

Variation	Use
Sparse transitions	Large alphabet
Count-only AC	Count matches without storing all
Dynamic AC	Advanced insertions

Variation	Use
AC + DP	Avoid forbidden substrings

16. Related Algorithms/Data Structures

- Trie: base structure.
- KMP: failure-link idea for one pattern.
- Suffix automaton: represents substrings of one text, different purpose.
- Rabin-Karp: can match many same-length patterns with hashes.

17. Practice Problems

Easy

Problem	Platform	Main Idea	Difficulty
Implement Trie	LeetCode	Foundation	Medium but basic
String Matching in an Array	LeetCode	Simpler multi-pattern	Easy

Medium

Problem	Platform	Main Idea	Difficulty
Multi-pattern Search	GFG	AC automaton	Medium
Stream of Characters	LeetCode	Reversed trie/AC idea	Hard-ish Medium

Hard

Problem	Platform	Main Idea	Difficulty
Censoring/Forbidden String DP	Codeforces	AC + DP	Hard
SPOJ Ada and Jobs-like pattern tasks	SPOJ	AC occurrence counting	Hard

18. Interview Explanation

"Aho-Corasick builds a trie of all patterns and adds failure links, similar to KMP fallback links. Then I scan the text once, moving through the automaton and reporting all patterns ending at each position. It is ideal when many patterns must be matched against one text."

19. Revision Notes

- Trie + failure links.
- BFS builds links.
- Inherit outputs from failure link.
- Scan is $O(\text{text} + \text{matches})$.
- Use sparse maps for large alphabets.

20. Final Cheat Sheet

Item	Note
Use when	Multiple exact patterns
Main operation	Automaton transition + output
Complexity	$O(\text{total pattern length} + \text{text} + \text{matches})$
Key code idea	Failure links like multi-pattern KMP
Edge cases	Suffix patterns, duplicates, unsupported chars

Palindromic Tree / Eertree

1. Overview

A palindromic tree, also called an Eertree, stores all distinct palindromic substrings of a string while processing characters online.

2. Intuition

Every new character can create at most one new distinct palindrome ending at the current position. Eertree keeps palindrome nodes and suffix links to quickly find the largest palindrome that can be extended.

3. When to Use It

Use Eertree when:

- need all distinct palindromic substrings
- process string online
- count occurrences of each distinct palindrome
- advanced CP palindrome problems
- Manacher gives radii but you need actual distinct palindrome structure

4. When Not to Use It

Avoid it when:

- You only need longest palindromic substring; use center expansion or Manacher.
- You only need palindrome yes/no; use two pointers.
- Placement interviews rarely expect full Eertree implementation.
- Simpler DP is accepted.

5. Core Concepts

Concept	Meaning	Why It Matters
Node	One distinct palindrome	Stores length and transitions

Concept	Meaning	Why It Matters
Suffix link	Longest proper palindromic suffix	Fast extension
Two roots	Length -1 and 0 roots	Handle odd/even uniformly
Current suffix palindrome	Longest pal suffix of processed prefix	Extension start

6. Step-by-Step Algorithm

For each added character:

1. Start from current longest palindromic suffix.
2. Follow suffix links until finding a palindrome that can be extended by the new character.
3. If the resulting palindrome already exists, move current pointer there.
4. Otherwise create a new node.
5. Set its suffix link by finding the next extendable suffix.
6. Increment occurrence count for current node.

7. Dry Run

String: **ababa**

Distinct palindromes created:

Step	Character	New Palindrome
1	a	a
2	b	b
3	a	aba
4	b	bab
5	a	ababa

Also existing palindrome **a** occurs again.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

class Eertree {
private:
    struct Node {
        int len = 0;
        int link = 0;
        long long occurrences = 0;
        map<char, int> next;
    };
};
```

```

vector<Node> tree;
string s;
int current;

int getSuffixCandidate(int node, int position, char ch) {
    while (true) {
        int length = tree[node].len;
        int mirrorPosition = position - 1 - length;
        if (mirrorPosition >= 0 && s[mirrorPosition] == ch) {
            return node;
        }
        node = tree[node].link;
    }
}

public:
    Eertree() {
        tree.resize(2);
        tree[0].len = -1;
        tree[0].link = 0;
        tree[1].len = 0;
        tree[1].link = 0;
        current = 1;
    }

    void addChar(char ch) {
        s.push_back(ch);
        int position = (int)s.size() - 1;

        int candidate = getSuffixCandidate(current, position, ch);

        if (tree[candidate].next.count(ch)) {
            current = tree[candidate].next[ch];
            tree[current].occurrences++;
            return;
        }

        int newNode = tree.size();
        tree.push_back(Node());
        tree[newNode].len = tree[candidate].len + 2;
        tree[candidate].next[ch] = newNode;

        if (tree[newNode].len == 1) {
            tree[newNode].link = 1;
        } else {
            int linkCandidate = getSuffixCandidate(tree[candidate].link, position,
ch);
            tree[newNode].link = tree[linkCandidate].next[ch];
        }

        current = newNode;
        tree[current].occurrences = 1;
    }
}

```

```

void build(const string& text) {
    for (char ch : text) addChar(ch);
}

int countDistinctPalindromes() const {
    return (int)tree.size() - 2;
}

};

int main() {
    Eertree et;
    et.build("ababa");
    cout << et.countDistinctPalindromes() << "\n";
    return 0;
}

```

pYTHON IMPLEMENTATION

```

class Eertree:
    def __init__(self):
        self.length = [-1, 0]
        self.link = [0, 0]
        self.next = [dict(), dict()]
        self.occurrences = [0, 0]
        self.s = []
        self.current = 1

    def _get_suffix_candidate(self, node: int, pos: int, ch: str) -> int:
        while True:
            length = self.length[node]
            mirror = pos - 1 - length
            if mirror >= 0 and self.s[mirror] == ch:
                return node
            node = self.link[node]

    def add_char(self, ch: str) -> None:
        self.s.append(ch)
        pos = len(self.s) - 1
        candidate = self._get_suffix_candidate(self.current, pos, ch)

        if ch in self.next[candidate]:
            self.current = self.next[candidate][ch]
            self.occurrences[self.current] += 1
            return

        new_node = len(self.length)
        self.length.append(self.length[candidate] + 2)
        self.link.append(0)
        self.next.append({})
        self.occurrences.append(1)
        self.next[candidate][ch] = new_node

```

```

        if self.length[new_node] == 1:
            self.link[new_node] = 1
        else:
            link_candidate = self._get_suffix_candidate(self.link[candidate], pos,
ch)
            self.link[new_node] = self.next[link_candidate][ch]

        self.current = new_node

    def build(self, s: str) -> None:
        for ch in s:
            self.add_char(ch)

    def count_distinct_palindromes(self) -> int:
        return len(self.length) - 2

```

10. Code Explanation

- Root `-1` handles odd-length palindromes cleanly.
- Root `0` handles even-length palindromes.
- `current` points to the longest palindromic suffix after processing the current prefix.
- `getSuffixCandidate` follows suffix links until the new character can extend a palindrome.
- Each non-root node corresponds to one distinct palindrome.

11. Complexity Analysis

Operation	Time	Space
Add one character	Amortized $O(\log \text{alphabet})$ with map, $O(1)$ average with unordered map	Up to one node
Build string	$O(n \log \text{alphabet})$ or $O(n)$ average	$O(n)$
Count distinct palindromes	$O(1)$	$O(1)$

12. Common Patterns

Pattern	Identify It	Approach	Examples
Distinct palindromes	Need unique pal substrings	Eertree nodes	CP palindrome tasks
Online palindrome processing	Add chars one by one	Eertree extend	Advanced CP
Occurrence counts	Frequency of each palindrome	Propagate counts by length	Codeforces tasks

13. Common Mistakes

- Forgetting the two special roots.
- Wrong suffix link for length-1 palindromes.
- Off-by-one in mirror position.
- Assuming every occurrence creates a new node.
- Not propagating occurrence counts if final frequencies are required.

14. Edge Cases

- Empty string.
- Single character.
- All same characters.
- No palindrome longer than 1.
- Repeated palindromes.
- Large alphabet.

15. Variations

Variation	Use
Basic Eertree	Count distinct palindromes
Occurrence Eertree	Count frequency of each palindrome
Double-ended Eertree	Advanced deque updates
Persistent Eertree	Advanced versioned queries

16. Related Algorithms/Data Structures

- Manacher: faster/simpler for radii and longest palindrome.
- DP palindrome table: simpler for small constraints.
- Suffix automaton: stores all substrings, not specifically palindromes.
- Trie: tree-like structure, but prefix-based.

17. Practice Problems

Easy

Problem	Platform	Main Idea	Difficulty
Palindromic Substrings	LeetCode	Simpler center expansion	Medium but prerequisite
Longest Palindromic Substring	LeetCode	Manacher/center	Medium

Medium

Problem	Platform	Main Idea	Difficulty
Count Different Palindromic Subsequences	LeetCode	DP, contrast topic	Hard but related
Distinct Palindromic Substrings	GFG	Eertree concept	Medium/Hard

Hard

Problem	Platform	Main Idea	Difficulty
Codeforces Eertree Problems	Codeforces	Distinct palindrome structure	Hard
SPOJ NUMOFPAL-like tasks	SPOJ	Palindromic tree/counting	Hard

18. Interview Explanation

"An Eertree stores each distinct palindromic substring as a node. While adding characters, it follows suffix links from the current longest palindromic suffix to find what can be extended. Each new character creates at most one new distinct palindrome, so the structure is linear in size."

19. Revision Notes

- Two roots: length -1 and 0 .
- One node per distinct palindrome.
- Current = longest palindromic suffix.
- Add char creates at most one new node.
- More CP than placement.

20. Final Cheat Sheet

Item	Note
Use when	Distinct palindromic substrings online
Main operation	Extend palindromic suffix
Complexity	$O(n)$ average
Key code idea	Suffix links over palindromes
Edge cases	Roots, single chars, all same chars